UNIVERSITÉ ALIOUNE DIOP DE BAMBEY

UFR Sciences Appliquées et Technologies de l'Information et de la Communication

Master 2 – Data Science

TP NOTÉ · DEVOPS / CONTENEURISATION

Déployer l'application microservices « Bijouterie » sur Kubernetes avec Jenkins et Argo CD

Namespaces · Deployments · Services · ConfigMaps · Secrets · Intégration continue · GitOps

Étudiant	GUEYE Mouhamadou Bassirou
Groupe	Travail individuel
Dépôt Git	https://github.com/mbg1984/bijouterie-microservices
Docker Hub	https://hub.docker.com/u/bassirou2024
Date de remise	17/11/2026

Année universitaire 2026 – 2027

Sommaire

1. Introduction : objectifs et architecture 3
1.1 Contexte et objectifs 3
1.2 Environnement de travail 3
1.3 Architecture de mon déploiement 4
2. Prise en main locale avec docker compose 4
3. Manifests Kubernetes 6
3.1 Organisation du dossier k8s/ 6
3.2 ConfigMap ou Secret ? 7
3.3 envFrom ou env[].valueFrom ? 7
3.4 Base de données MySQL 9
3.5 DNS interne du cluster et types de Services 9
4. Tests du déploiement Kubernetes 13
5. Pipeline Jenkins 17
5.1 Installation de Jenkins 17
5.2 Credentials 17
5.3 Configuration du job 18
5.4 Le Jenkinsfile commenté 19
5.5 Résultat des builds 22
6. Argo CD : déploiement continu GitOps 23
6.1 Installation 23
6.2 L'Application bijouterie 24
6.3 Démonstrations 25
7. Sécurité : la gestion des Secrets dans Git 27
Ce qu'il faudrait faire en production 27
Bonnes pratiques appliquées dans ce TP 28
8. Difficultés rencontrées et solutions 28
9. Conclusion 29
Pistes d'amélioration 29

1. Introduction : objectifs et architecture

1.1 Contexte et objectifs

Une bijouterie souhaite mettre en ligne son catalogue de bijoux en or. L'application fournie est découpée en trois microservices déjà conteneurisés : un **frontend** Nginx qui sert le site et relaie les appels `/api/*`, un **backend** Node.js/Express qui expose une API REST CRUD, et une base **MySQL 8.4** qui stocke les bijoux et leurs photos.

En tant qu'équipe DevOps, ma mission était d'industrialiser le déploiement sans modifier le code applicatif :

- écrire les **manifests Kubernetes** en externalisant toute la configuration dans des ConfigMaps et des Secrets ;
- mettre en place l'**intégration continue avec Jenkins** : construction, double tag et publication des images sur Docker Hub ;
- mettre en place le **déploiement continu GitOps avec Argo CD** : Git devient la seule source de vérité ;
- documenter et prouver chaque étape par des captures commentées.

1.2 Environnement de travail

Composant	Version / choix	Rôle
Poste	Windows 11 Pro, 16 Go de RAM	Machine de développement
Docker Desktop	27.3.1 (moteur Hyper-V, 4 Go / 4 CPU)	Moteur de conteneurs
Cluster	kind v0.33.0 – Kubernetes v1.37.0	Cluster local à 1 nœud
kubectl	fourni par Docker Desktop	Client Kubernetes
Jenkins	LTS 2.580 (JDK 21) dans un conteneur	Intégration continue
Argo CD	manifests « stable », namespace `argocd`	Déploiement continu GitOps
Registre	Docker Hub – compte `bassirou2024`	Stockage des images
Dépôt	GitHub – `mbg1984/bijouterie-microservices`	Source de vérité

J'ai choisi **kind** (Kubernetes in Docker) car il crée un vrai cluster Kubernetes dans un conteneur Docker, se recrée en une commande et permet d'exposer le NodePort 30080 sur `localhost` grâce à `extraPortMappings` :

kind-config.yaml

```yaml
# Cluster kind pour le TP Bijouterie
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    # Expose le NodePort 30080 du nœud sur localhost:30080 (accès au frontend)
    extraPortMappings:
      - containerPort: 30080
        hostPort: 30080
        protocol: TCP
```

1.3 Architecture de mon déploiement

Le schéma ci-dessous, réalisé pour ce rapport, résume l'ensemble : en haut la chaîne CI/CD, en bas le cluster.

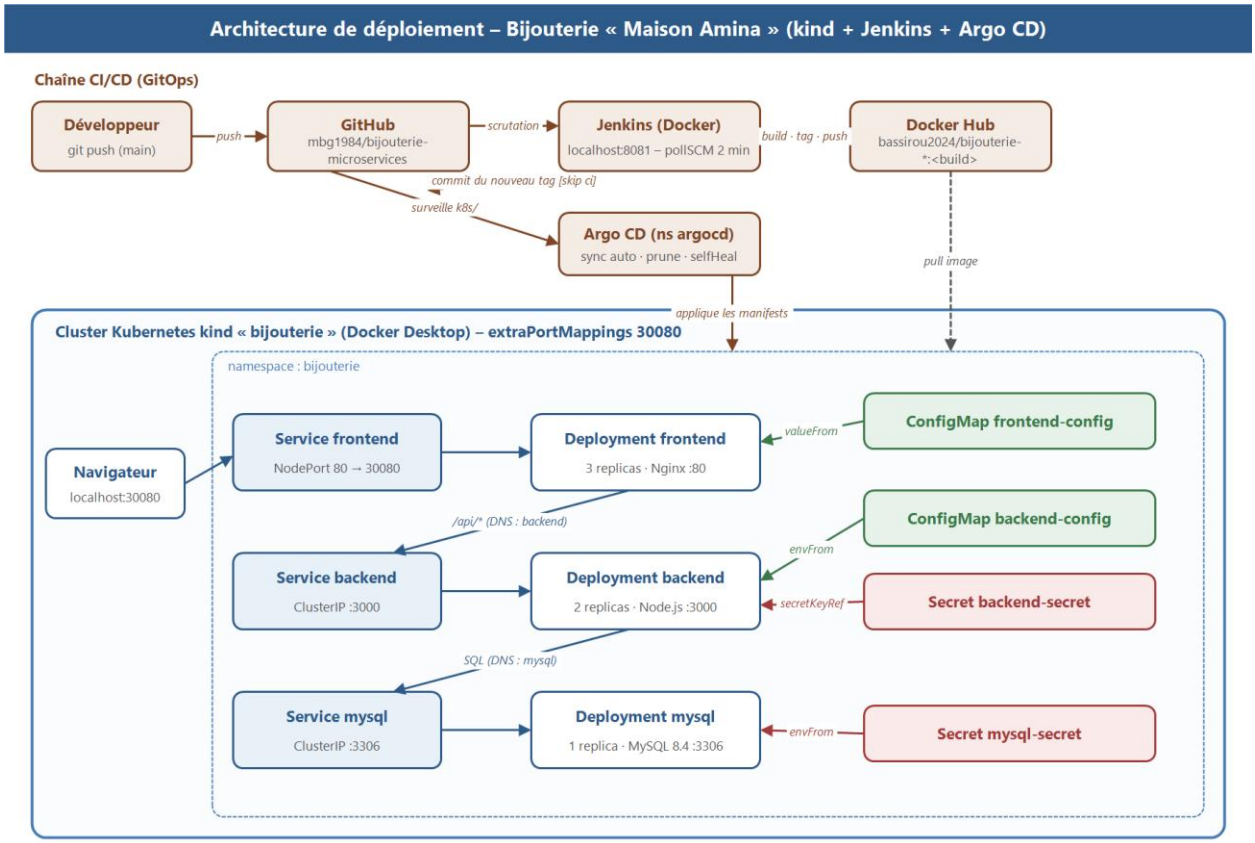


Figure 1 – Schéma personnel de l'architecture déployée

Ce que montre la capture : la chaîne Développeur → GitHub → Jenkins → Docker Hub, Argo CD qui surveille le dossier `k8s/`, et dans le namespace `bijouterie` les trois couples Service/Deployment avec leurs ConfigMaps et Secrets. **Ce qu'elle prouve :** la compréhension globale du flux : le navigateur ne parle qu'au frontend (NodePort 30080), le frontend joint le backend par le nom DNS `backend`, le backend joint MySQL par le nom DNS `mysql`.

2. Prise en main locale avec docker compose

Avant Kubernetes, j'ai lancé l'application telle que livrée :

Commandes utilisées

```
cp .env.example .env
docker compose up --build -d
# puis http://localhost:8082
```

Adaptation : le port 8080 prévu par défaut était déjà occupé sur mon poste par un processus Java (`Get-NetTCPConnection -LocalPort 8080` → PID 10832, `java`). Le `docker-compose.yml` prévoit la variable `FRONTEND_PORT` : je l'ai fixée à `8082` dans mon fichier `.env` (non versionné), sans toucher au code.

Variables d'environnement repérées

Service	Variables	Sensibles ?

mysql	`MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE`, `MYSQL_USER`, `MYSQL_PASSWORD`	oui (mots de passe, utilisateur)
backend	`PORT`, `DB_HOST`, `DB_PORT`, `DB_NAME`, `SHOP_NAME`, `CURRENCY`, `MAX_FILE_SIZE_MB`	non
backend	`DB_USER`, `DB_PASSWORD`	oui
frontend	`BACKEND_URL`, `MAX_UPLOAD_SIZE`	non

Comment le frontend joint-il le backend ? Le fichier `frontend/nginx/default.conf.template` contient `proxy_pass ${BACKEND_URL};`. Au démarrage, l'image officielle Nginx exécute `envsubst` sur ce template et remplace `${BACKEND_URL}` par la valeur de la variable d'environnement (ici `http://backend:3000`). Toutes les requêtes `/api/*` du navigateur sont ainsi relayées au backend, qui n'a pas besoin d'être exposé.

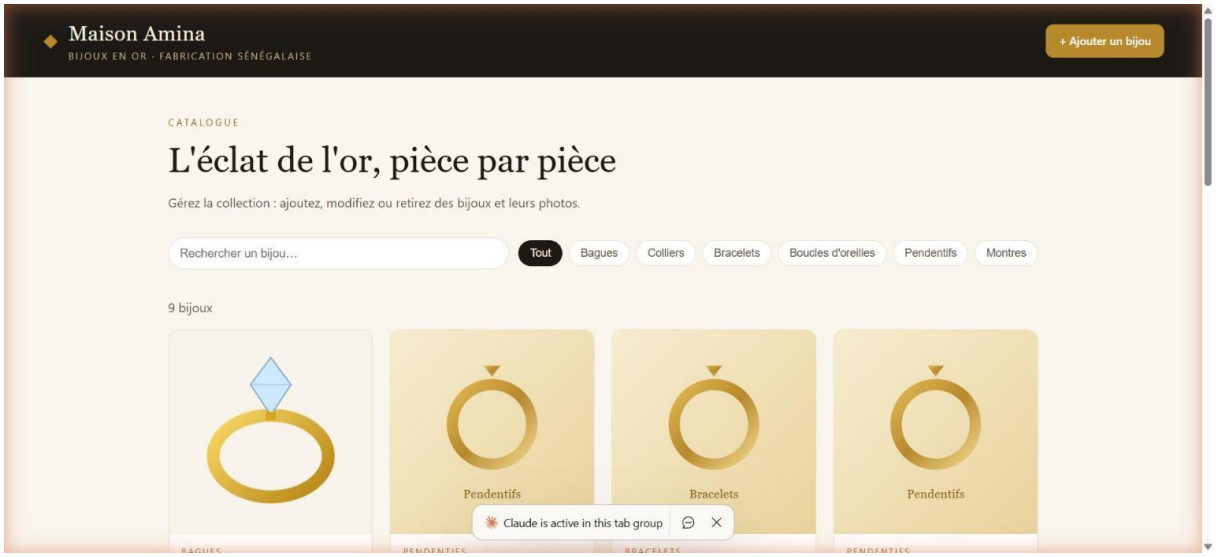


Figure 2 – Application lancée avec docker compose (http://localhost:8082)

Ce que montre la capture : le catalogue « Maison Amina » avec 9 bijoux, dont la « Bague Solitaire Amina » ajoutée avec sa photo. **Ce qu'elle prouve :** les trois microservices fonctionnent ensemble en local : upload de photo, stockage MySQL et affichage.

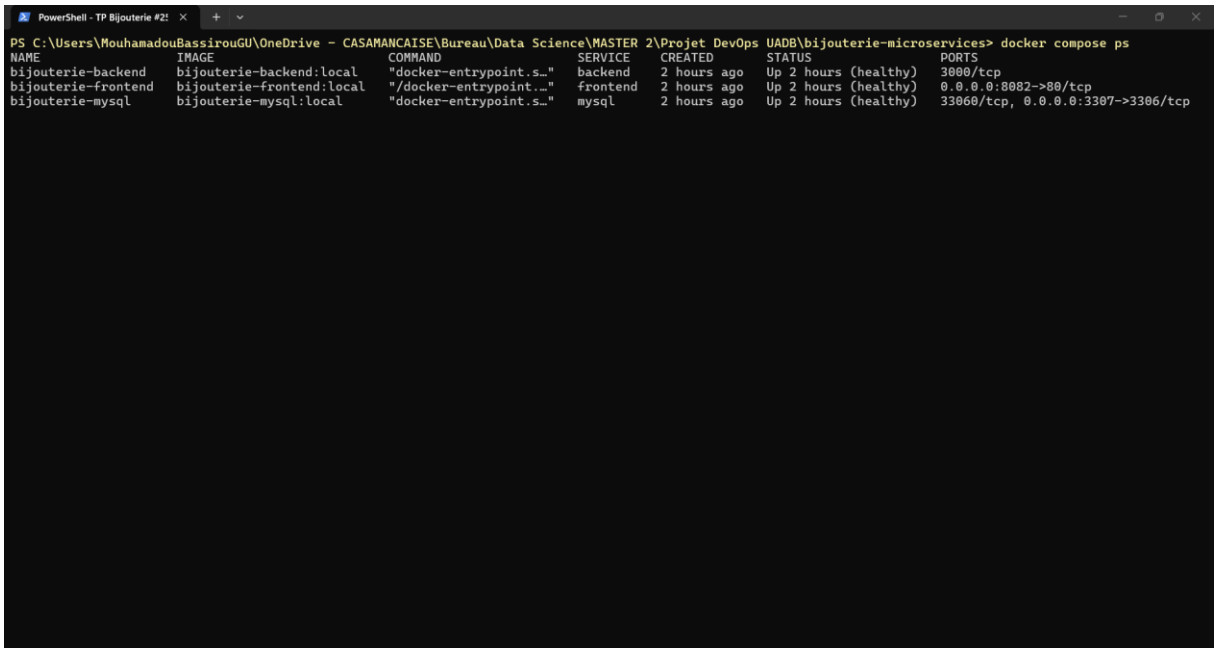


Figure 3 – Sortie de docker compose ps

Ce que montre la capture : les conteneurs `bijouterie-frontend`, `bijouterie-backend` et `bijouterie-mysql` à l'état `Up (healthy)` ; le frontend est publié sur `0.0.0.0:8082->80`. **Ce qu'elle prouve :** les healthchecks des Dockerfiles passent ; seul le frontend (et MySQL pour inspection) est exposé.

3. Manifests Kubernetes

3.1 Organisation du dossier k8s/

J'ai conservé **un fichier par ressource**, avec le préfixe numérique imposé. Je n'ai pas regroupé de ressources avec `---` : un fichier par objet rend les diffs Git et l'historique Argo CD plus lisibles, et l'ordre alphabétique (`00-` → `10-`) crée le namespace avant le reste lors d'un `kubectl apply -f k8s/`.

Fichier	Ressource	Points clés
`00-namespace.yaml`	Namespace `bijouterie`	isole toutes les ressources de l'application
`01-mysql-secret.yaml`	Secret `mysql-secret`	4 clés MYSQL_* en base64
`02-mysql-deployment.yaml`	Deployment `mysql`	1 replica, stratégie `Recreate`, `envFrom` secretRef
`03-mysql-service.yaml`	Service `mysql`	ClusterIP 3306 : base non exposée hors du cluster
`04-backend-configmap.yaml`	ConfigMap `backend-config`	7 clés non sensibles
`05-backend-secret.yaml`	Secret `backend-secret`	DB_USER / DB_PASSWORD = MYSQL_USER / MYSQL_PASSWORD
`06-backend-deployment.yaml`	Deployment `backend`	2 replicas, `envFrom` + `secretKeyRef`, sondes
`07-backend-service.yaml`	Service `backend`	ClusterIP 3000
`08-frontend-configmap.yaml`	ConfigMap `frontend-config`	BACKEND_URL, MAX_UPLOAD_SIZE
`09-frontend-deployment.yaml`	Deployment `frontend`	2 puis 3 replicas, `configMapKeyRef`
`10-frontend-service.yaml`	Service `frontend`	NodePort 80 → 30080

Contraintes communes respectées : `metadata.namespace: bijouterie` explicite dans chaque fichier, labels `app: bijouterie` + `tier: frontend|backend|database` sur toutes les ressources, `selector` des Services et Deployments basés sur ces deux labels, images `bassirou2024/bijouterie-*:<numéro de build>` (jamais `latest`), `resources.requests` / `limits` sur chaque conteneur et commentaires `#` sur les champs importants.

3.2 ConfigMap ou Secret ?

Une **ConfigMap** contient la configuration non sensible (port, nom de la boutique, devise, URL interne). Un **Secret** contient ce qui donnerait un accès s'il fuyait : identifiants et mots de passe MySQL. Les valeurs d'un Secret sont encodées en base64 (`echo -n 'valeur' | base64`). Les identifiants du backend (`backend-secret`) sont **identiques** à ceux créés par MySQL (`mysql-secret`) : sinon MySQL répond « Access denied ». Attention, MySQL ne crée l'utilisateur qu'au **premier** démarrage de la base.

k8s/04-backend-configmap.yaml

```yaml
# ================================================================================
# ConfigMap « backend-config » : configuration NON sensible du backend
# --------------------------------------------------------------------------------
# Injectée en bloc dans le conteneur backend avec envFrom : chaque clé
# devient une variable d'environnement du même nom.
# Toutes les valeurs sont des chaînes (d'où les guillemets autour des nombres).
# ================================================================================
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
  namespace: bijouterie
  labels:
    app: bijouterie
    tier: backend
data:
  PORT: "3000"              # port d'écoute de l'API Node.js
  DB_HOST: "mysql"          # = nom du Service MySQL, résolu par le DNS du cluster
                            #   (mysql.bijouterie.svc.cluster.local)
  DB_PORT: "3306"           # port du Service MySQL
  DB_NAME: "bijouterie"     # doit correspondre à MYSQL_DATABASE
  SHOP_NAME: "Maison Amina Joaillerie" # nom de la boutique affiché en en-tête du site
  CURRENCY: "FCFA"          # devise des prix
  MAX_FILE_SIZE_MB: "5"     # taille max d'une photo (Mo), contrôlée par l'API
```

k8s/05-backend-secret.yaml (valeurs masquées dans ce rapport)

```yaml
# ================================================================================
# Secret « backend-secret » : identifiants MySQL utilisés par le backend
# --------------------------------------------------------------------------------
# Les valeurs DOIVENT être identiques à MYSQL_USER / MYSQL_PASSWORD de
# mysql-secret, sinon MySQL répond « Access denied » au backend.
# Injecté clé par clé dans le Deployment backend (env[].valueFrom.secretKeyRef).
# ================================================================================
apiVersion: v1
kind: Secret
metadata:
  name: backend-secret
  namespace: bijouterie
  labels:
    app: bijouterie
    tier: backend
type: Opaque
data:
  DB_USER: <valeur base64 masquée>                      # "amina"
  DB_PASSWORD: <valeur base64 masquée>     # même valeur que MYSQL_PASSWORD
```

3.3 envFrom ou env[].valueFrom ?

Méthode	Effet	Utilisée dans

`envFrom` (configMapRef / secretRef)	importe **toutes** les clés de la ressource ; chaque clé devient une variable du même nom	mysql (`mysql-secret`), backend (`backend-config`)
`env[].valueFrom` (configMapKeyRef / secretKeyRef)	importe **une seule** clé, choisie explicitement, sous un nom de variable choisi	backend (`DB_USER`, `DB_PASSWORD`), frontend (`BACKEND_URL`, `MAX_UPLOAD_SIZE`)

`envFrom` est concis quand toutes les clés servent ; `valueFrom` est plus explicite et plus sûr pour les secrets (on n'injecte que ce qui est nécessaire, et on peut renommer la variable). Si une clé référencée manque, le pod reste en `CreateContainerConfigError`.

k8s/06-backend-deployment.yaml (extrait commenté complet)

```yaml
# ================================================================================
# Deployment « backend » : API REST Node.js / Express
# --------------------------------------------------------------------------------
# 2 replicas : le backend est sans état (les photos sont stockées dans MySQL),
# on peut donc le répliquer. Le Service « backend » répartit les requêtes.
# ================================================================================
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: bijouterie
  labels:
    app: bijouterie
    tier: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: bijouterie
      tier: backend
  template:
    metadata:
      labels:
        app: bijouterie
        tier: backend
    spec:
      containers:
        - name: backend
          image: bassirou2024/bijouterie-backend:8
          ports:
            - name: http
              containerPort: 3000
          # Méthode 1 : envFrom -> toutes les clés de backend-config
          # (PORT, DB_HOST, DB_PORT, DB_NAME, SHOP_NAME, CURRENCY, MAX_FILE_SIZE_MB)
          envFrom:
            - configMapRef:
                name: backend-config
          # Méthode 2 : env[].valueFrom -> une variable = une clé précise,
          # en choisissant explicitement le nom de la variable
          env:
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: backend-secret
                  key: DB_USER
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: backend-secret
                  key: DB_PASSWORD
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 256Mi
          # Readiness : /api/ready renvoie 503 tant que MySQL n'est pas joignable
```

```
          # -> le pod est retiré des endpoints du Service, sans être redémarré
          readinessProbe:
            httpGet:
              path: /api/ready
              port: http
            initialDelaySeconds: 5
            periodSeconds: 5
          # Liveness : /api/health vérifie seulement que le processus répond
          # -> en cas d'échec, kubelet redémarre le conteneur
          livenessProbe:
            httpGet:
              path: /api/health
              port: http
            initialDelaySeconds: 15
            periodSeconds: 10
```

3.4 Base de données MySQL

Un seul replica, car deux pods MySQL auraient chacun leurs propres données. La stratégie `Recreate` évite d'avoir deux MySQL en même temps pendant une mise à jour. La **readinessProbe** `tcpSocket:3306` ne marque le pod prêt que lorsque MySQL écoute réellement (pendant l'initialisation, `init.sql` s'exécute sans réseau). Pas de volume persistant : `/var/lib/mysql` est dans la couche du conteneur, donc **effacé à chaque recréation** du pod (voir test 5).

3.5 DNS interne du cluster et types de Services

Chaque Service reçoit une IP virtuelle stable (ClusterIP) et un nom DNS géré par **CoreDNS** : `<service>.<namespace>.svc.cluster.local`. Depuis un pod du même namespace, le nom court suffit. C'est pourquoi `DB_HOST: mysql` et `BACKEND_URL: http://backend:3000` fonctionnent : le nom est résolu vers l'IP du Service, qui répartit ensuite le trafic vers les pods dont les labels correspondent au `selector` (liste des endpoints).

Service	Type	Pourquoi
`mysql`	ClusterIP :3306	accessible uniquement depuis le cluster (sécurité de la base)
`backend`	ClusterIP :3000	seul le frontend l'appelle ; répartit les requêtes entre les 2 pods
`frontend`	NodePort 80 → 30080	ouvre le port 30080 sur le nœud ; kind le mappe sur localhost:30080

k8s/10-frontend-service.yaml

```yaml
# ============================================================================
# Service « frontend » (NodePort)
# ----------------------------------------------------------------------------
# NodePort = ouvre le port 30080 sur chaque nœud du cluster et le redirige
# vers les pods frontend. Avec kind, le port 30080 du nœud est mappé sur
# localhost:30080 (extraPortMappings) -> site accessible sur
# http://localhost:30080
# ============================================================================
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: bijouterie
  labels:
    app: bijouterie
    tier: frontend
spec:
  type: NodePort
  selector:
    app: bijouterie
    tier: frontend
  ports:
    - name: http
      port: 80               # port du Service à l'intérieur du cluster
      targetPort: http       # port 80 du conteneur Nginx
      nodePort: 30080        # port ouvert sur le nœud (plage 30000-32767)
```

Déploiement initial (avant Argo CD) : les images taguées `1` ont été construites localement puis chargées dans le nœud avec `kind load docker-image`, et les manifests appliqués avec `kubectl apply -f k8s/`. Dès que Jenkins a publié les images sur Docker Hub, c'est Argo CD qui a pris le relais.

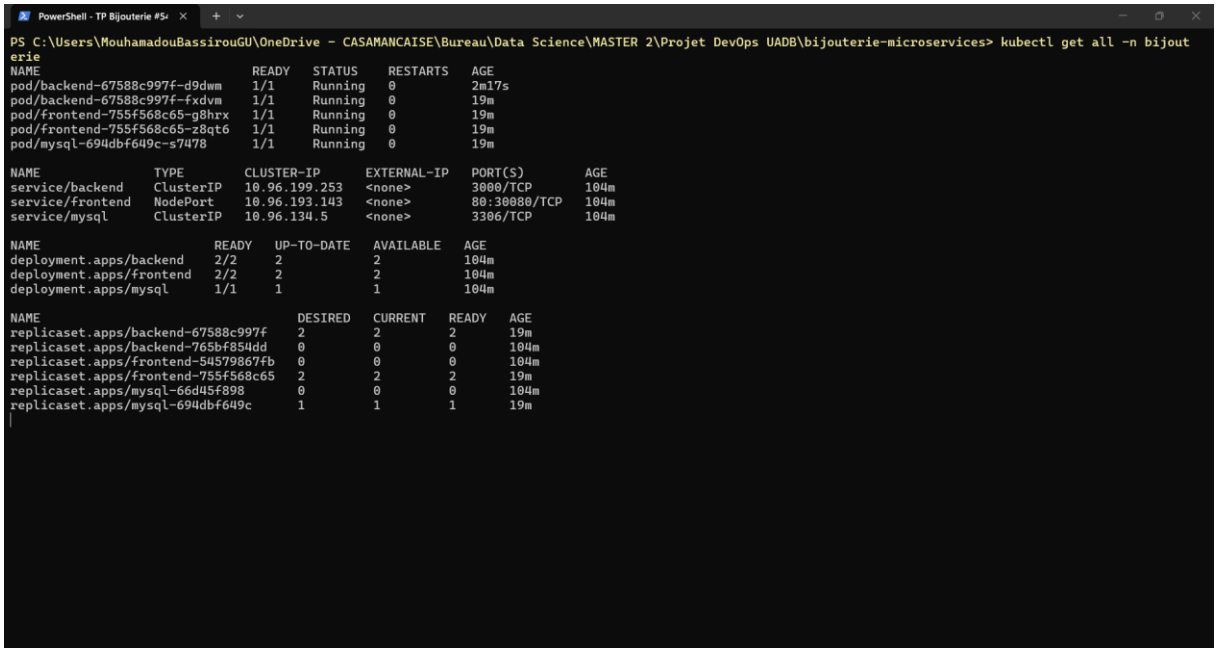


Figure 4 – kubectl get all -n bijouterie

Ce que montre la capture : 5 pods `Running` (2 backend, 2 frontend, 1 mysql), les 3 Services, les 3 Deployments et leurs ReplicaSets. **Ce qu'elle prouve :** le déploiement complet est opérationnel ; les anciens ReplicaSets à 0 témoignent des mises à jour successives.

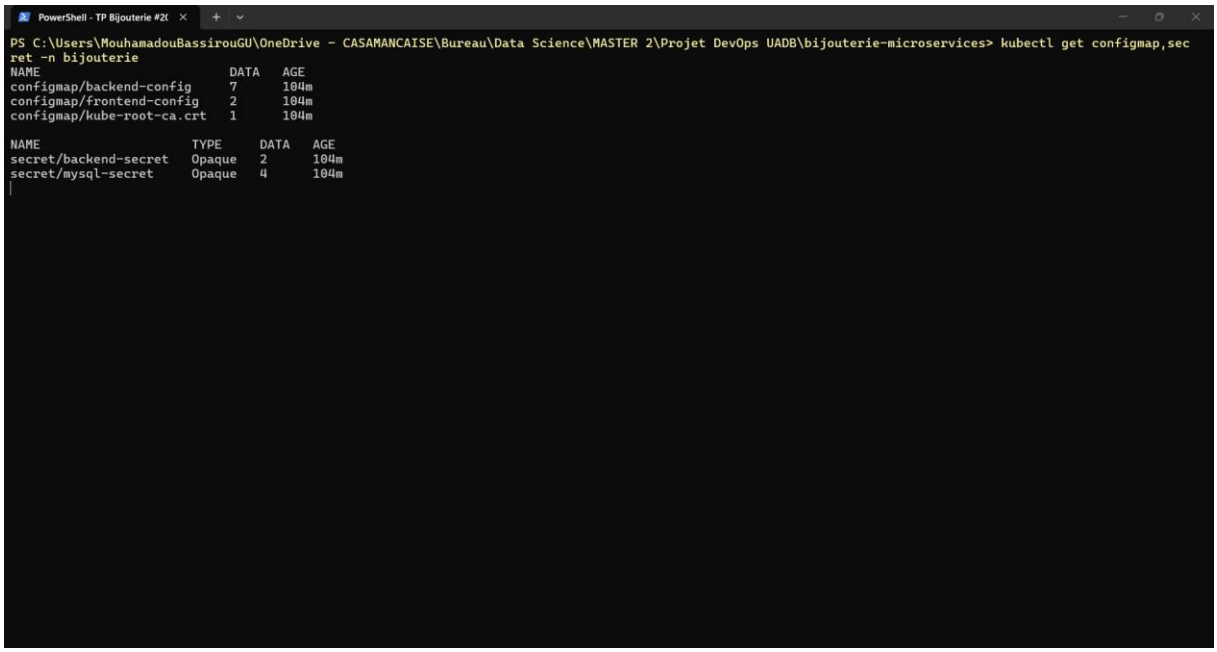


Figure 5 – kubectl get configmap,secret -n bijouterie

Ce que montre la capture : les ConfigMaps `backend-config` (7 clés) et `frontend-config` (2 clés), les Secrets `backend-secret` (2 clés) et `mysql-secret` (4 clés), de type Opaque. **Ce qu'elle prouve :** les noms et nombres de clés imposés par l'énoncé sont respectés ; aucune valeur secrète n'est affichée.



Figure 6 – kubectl describe configmap backend-config

Ce que montre la capture : le contenu de la ConfigMap : `DB_HOST: mysql`, `DB_PORT: 3306`, `SHOP_NAME`, `CURRENCY: FCFA`… **Ce qu'elle prouve :** la configuration est bien externalisée hors du Deployment.

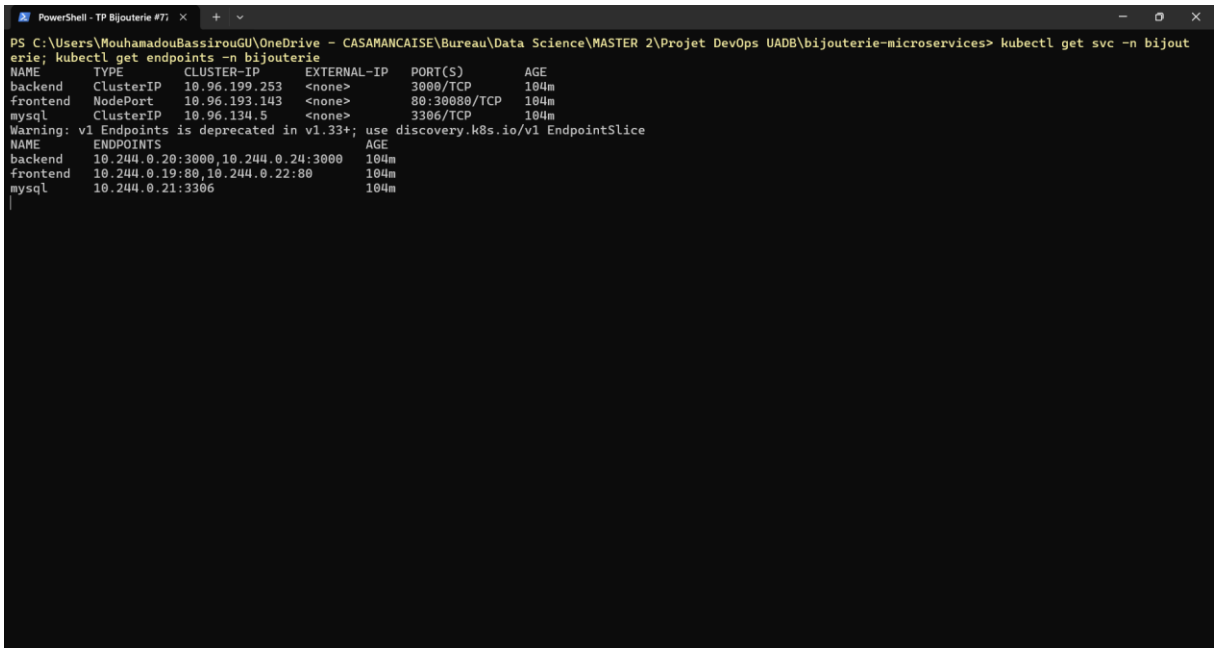


Figure 7 – kubectl get svc et get endpoints

Ce que montre la capture : le Service `frontend` en NodePort `80:30080/TCP`, `backend` et `mysql` en ClusterIP, et les endpoints (2 IP de pods derrière backend, 2 ou 3 derrière frontend, 1 derrière mysql). **Ce qu'elle prouve :** les `selector` trouvent bien les pods : chaque Service a des endpoints.

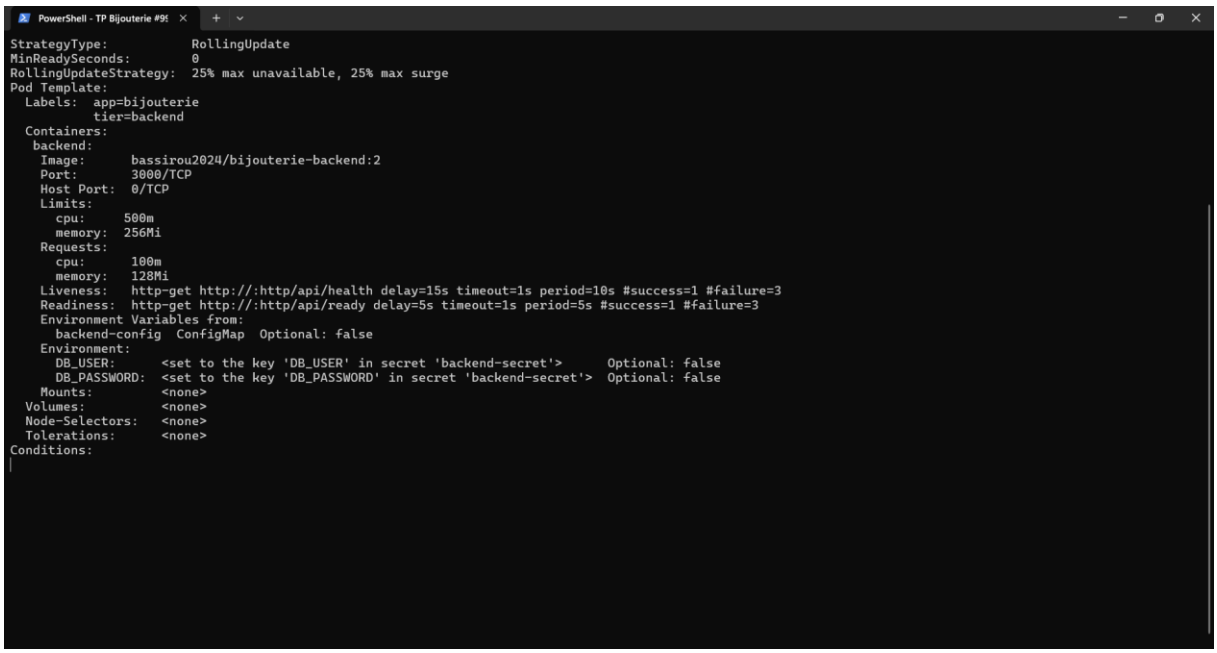


Figure 8 – kubectl describe deployment backend (extrait)

Ce que montre la capture : l'image taguée, les limits/requests, les sondes liveness (`/api/health`) et readiness (`/api/ready`), `backend-config ConfigMap` en *Environment Variables from* et `DB_USER`/`DB_PASSWORD` issus de `backend-secret`. **Ce qu'elle prouve :** les deux méthodes d'injection (envFrom et valueFrom) sont réellement appliquées.

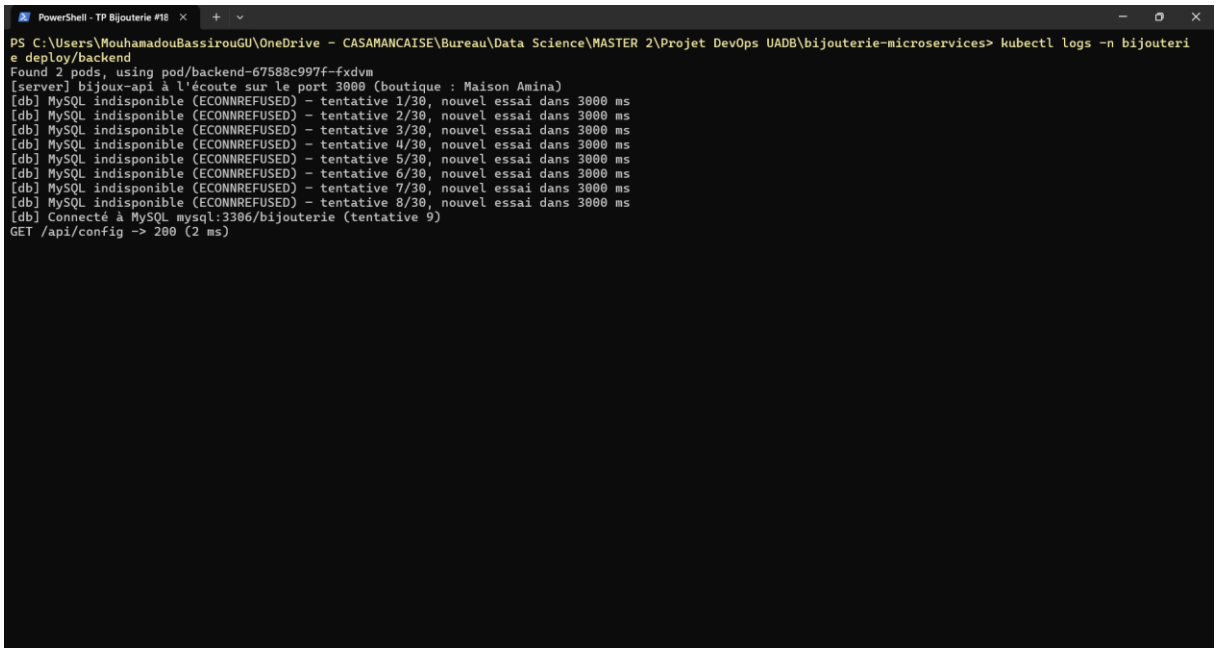


Figure 9 – kubectl logs -n bijouterie deploy/backend

Ce que montre la capture : les logs de démarrage du backend. **Ce qu'elle prouve :** le backend s'est connecté à MySQL via le Service `mysql`.

4. Tests du déploiement Kubernetes

#	Test (§ 4.9)	Résultat obtenu
1	Site via le NodePort avec le nom de la boutique	OK : http://localhost:30080 affiche « Maison Amina » (HTTP 200)
2	Ajout avec photo, modification, suppression	OK : création (id 9, `has_photo: true`), modification puis suppression (HTTP 204)
3	Suppression d'un pod backend	OK : un nouveau pod est recréé en ~3 s, prêt en ~20 s
4	Rechargements de /api/config	OK : `servedBy` alterne entre les 2 pods (3 / 5 réponses sur 8)
5	Suppression du pod MySQL	Les bijoux ajoutés disparaissent : retour aux 8 bijoux de démo

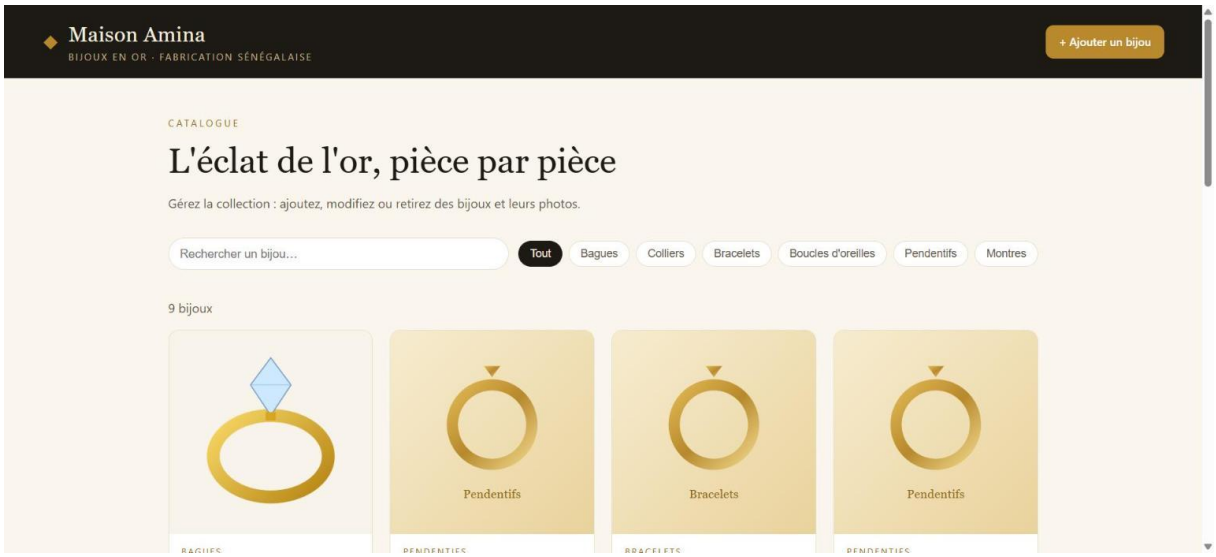


Figure 10 – Le site servi par Kubernetes (http://localhost:30080)

Ce que montre la capture : la page d'accueil avec l'en-tête « Maison Amina » et la devise FCFA, servie via le Service NodePort. **Ce qu'elle prouve :** la chaîne NodePort → frontend → backend → MySQL fonctionne dans le cluster kind.

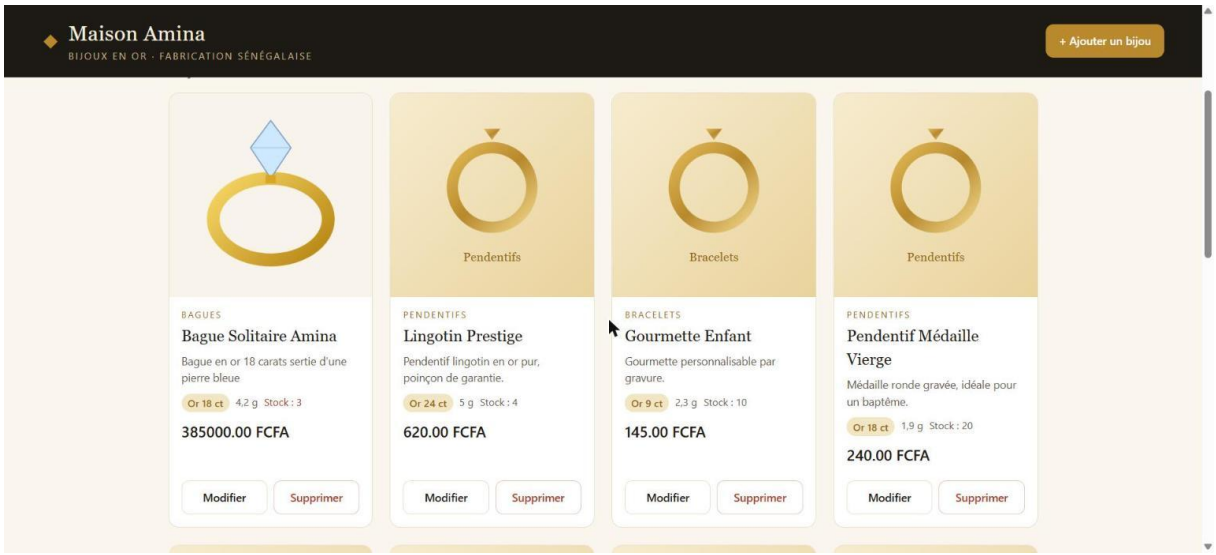


Figure 11 – Bijou ajouté avec sa photo

Ce que montre la capture : la carte « Bague Solitaire Amina » (or 18 ct, 4,2 g, 385 000 FCFA) avec sa photo. **Ce qu'elle prouve :** l'upload multipart traverse Nginx (`client_max_body_size`) puis le backend, et la photo est stockée en LONGBLOB.

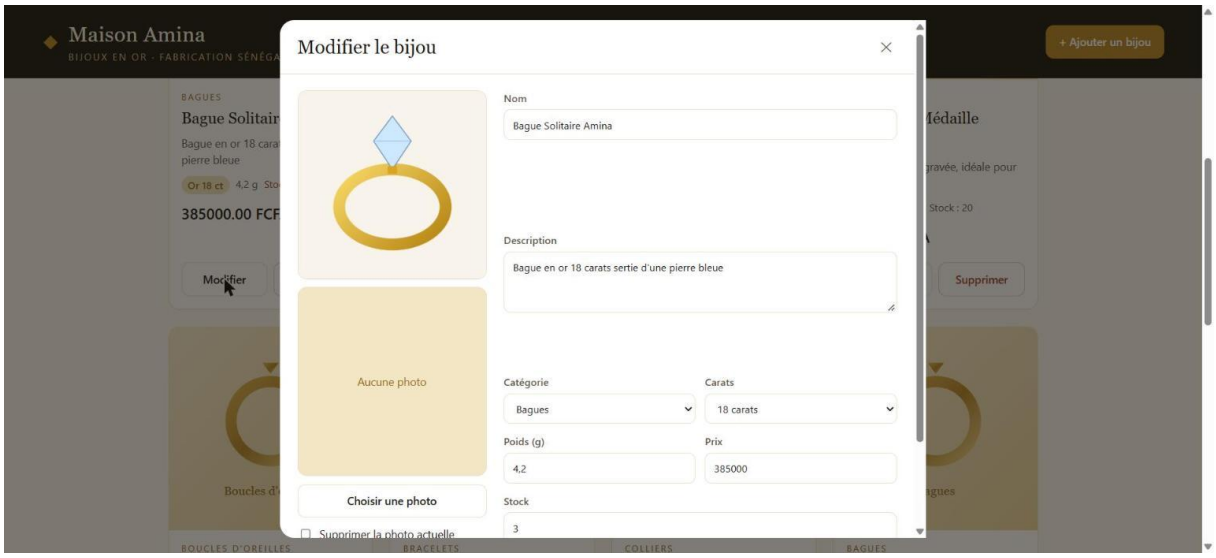


Figure 12 – Formulaire de modification du bijou

Ce que montre la capture : le formulaire « Modifier le bijou » pré-rempli avec la photo actuelle. **Ce qu'elle prouve :** la lecture d'un bijou et de sa photo (`GET /api/bijoux/:id/photo`) fonctionne.

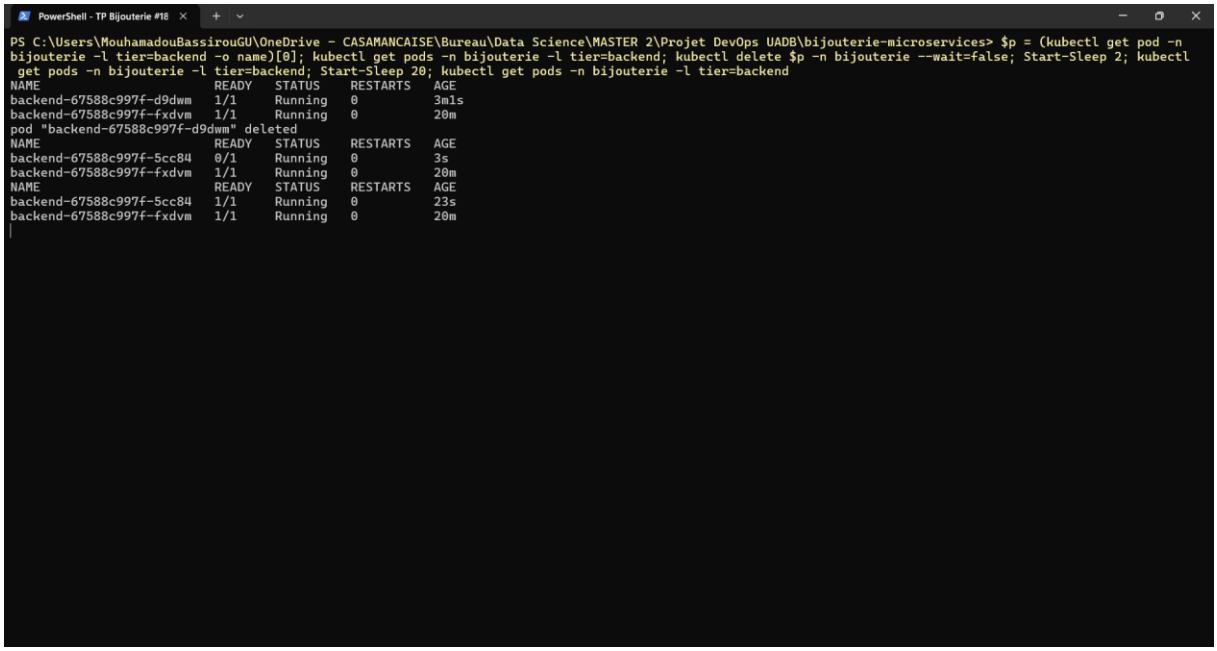


Figure 13 – Recréation automatique d'un pod supprimé

Ce que montre la capture : le pod `backend-…-d9dwm` est supprimé ; 2 s plus tard un nouveau pod `…-5cc84` est en `0/1`, puis `1/1` 20 s après. **Ce qu'elle prouve :** le ReplicaSet maintient en permanence le nombre de replicas demandé (2) : c'est l'auto-réparation de Kubernetes.

Rôle du Service ClusterIP (test 4)

La route `/api/config` renvoie `servedBy`, le nom du pod qui a répondu. En rechargeant, la valeur alterne entre les deux pods backend : le Service `backend` (via kube-proxy) répartit les connexions entre tous les endpoints prêts. Si un pod tombe ou n'est pas prêt (`/api/ready` en 503), il est retiré des endpoints et ne reçoit plus de trafic.

Pourquoi les données MySQL sont perdues (test 5)

Le Deployment MySQL n'a aucun volume : les fichiers de la base vivent dans la couche inscriptible du conteneur. Quand le pod est supprimé, Kubernetes en crée un **nouveau** avec une couche vide ; l'image

ré-exécute `init.sql` et on retrouve les 8 bijoux de démonstration. Pour conserver les données, il faudrait un PersistentVolumeClaim monté sur `/var/lib/mysql` (bonus non réalisé).

Changement du SHOP_NAME (capture 7)

Une variable d'environnement n'est lue qu'au démarrage du conteneur : modifier la ConfigMap ne suffit pas. Après le changement (fait par Git et appliqué par Argo CD, voir § 6), j'ai lancé `kubectl rollout restart deployment/backend -n bijouterie` : de nouveaux pods ont lu la nouvelle valeur.

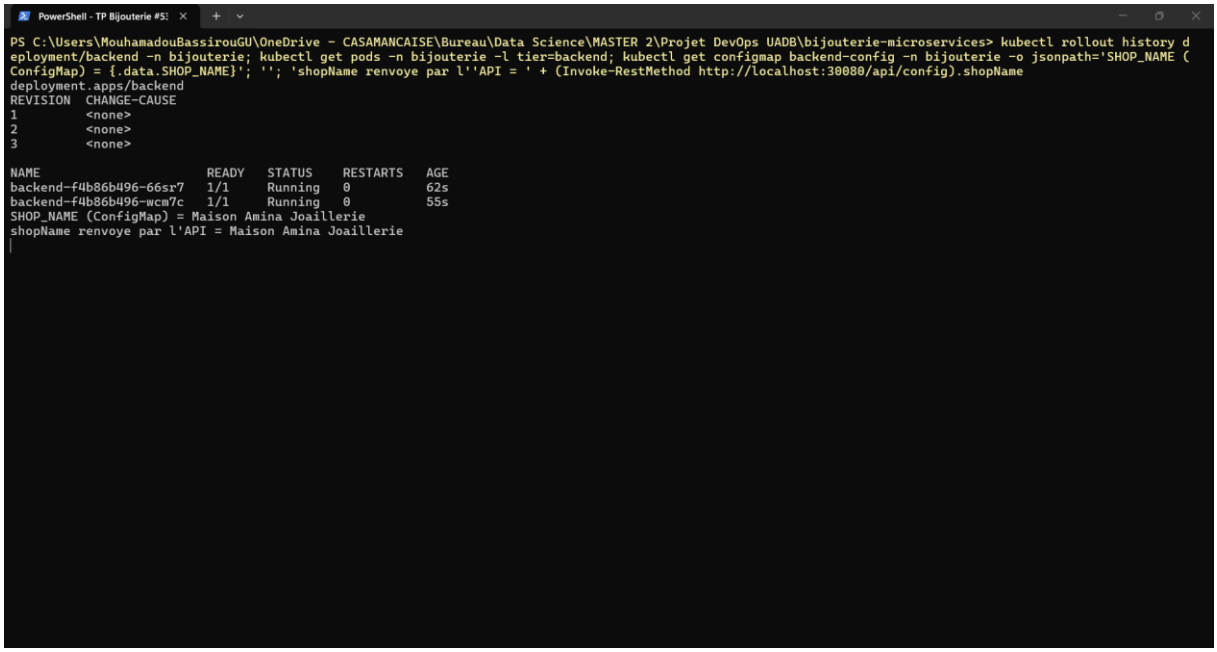


Figure 14 – SHOP_NAME modifié visible après rollout restart

Ce que montre la capture : l'historique de rollout du backend (révision 3), les nouveaux pods, la ConfigMap et l'API qui renvoient « Maison Amina Joaillerie ». **Ce qu'elle prouve :** les pods redémarrés ont bien relu la ConfigMap.

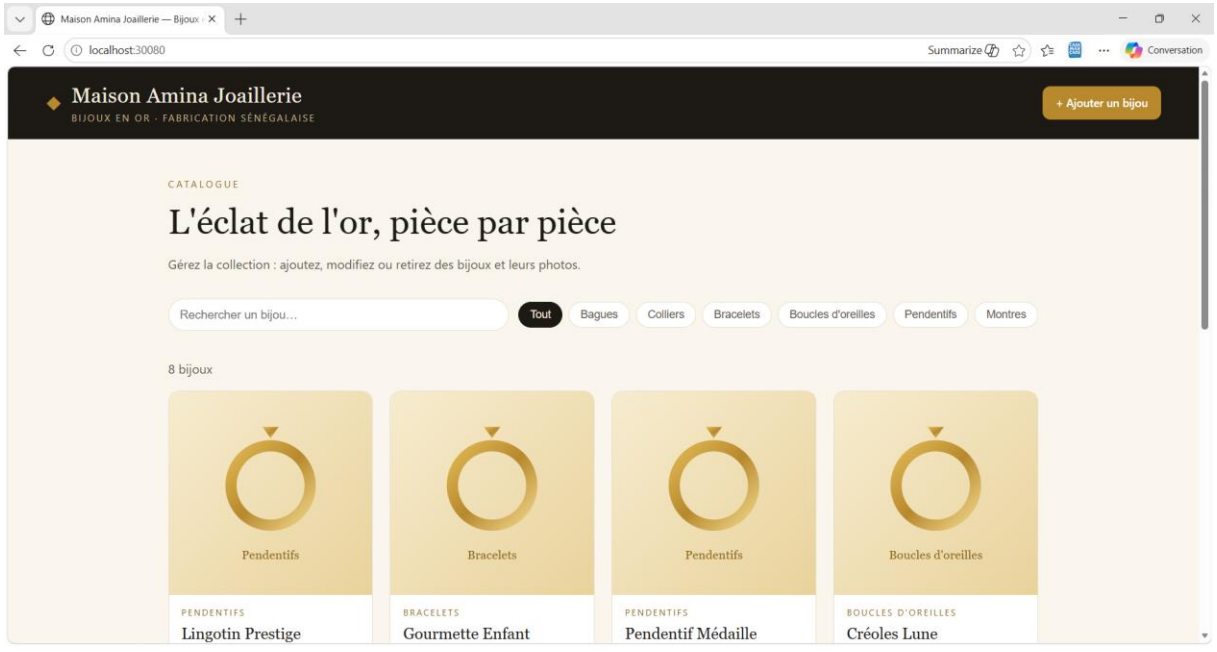


Figure 15 – En-tête du site avec le nouveau nom

Ce que montre la capture : le site http://localhost:30080 affichant « Maison Amina Joaillerie ». **Ce qu'elle prouve :** le changement de configuration est visible par l'utilisateur final.

5. Pipeline Jenkins

5.1 Installation de Jenkins

Jenkins tourne dans un conteneur Docker construit à partir de `jenkins/jenkins:lts-jdk21`, auquel j'ai ajouté le client Docker (`docker-ce-cli`) et les plugins du pipeline. Le socket `/var/run/docker.sock` de Docker Desktop est monté dans le conteneur (avec `group_add: 0` pour l'accès) : l'agent Jenkins peut donc exécuter `docker build/push`. Ce choix évite aussi le problème des étapes `sh` qui n'existent pas sous Windows.

jenkins/docker-compose.yml

```
# ============================================================================
# Lancement de Jenkins en local
#   docker compose up -d --build      -> http://localhost:8081
# ============================================================================
services:
  jenkins:
    build: .
    image: jenkins-bijouterie:local
    container_name: jenkins
    restart: unless-stopped
    ports:
      - "8081:8080"     # interface web (8080 est déjà pris par l'appli en local)
      - "50000:50000"   # agents Jenkins
    environment:
      # Mémoire limitée : Docker Desktop n'a que 4 Go à partager avec kind
      JAVA_OPTS: "-Xmx768m"
    group_add:
      - "0"             # groupe propriétaire du socket Docker -> accès à docker
    volumes:
      - jenkins_home:/var/jenkins_home                # config, jobs, credentials
      - /var/run/docker.sock:/var/run/docker.sock     # moteur Docker de l'hôte

volumes:
  jenkins_home:
```

5.2 Credentials

Sur Docker Hub, j'ai généré un **Access Token** (Lecture et écriture, expiration 30 jours) au lieu d'utiliser mon mot de passe. Pour le stage bonus, j'ai créé un **token GitHub à granularité fine**, limité au seul dépôt `bijouterie-microservices` avec la permission *Contents : Read and write*. Les deux sont stockés dans Jenkins comme credentials *Username with password* : `dockerhub-credentials` et `github-credentials`.

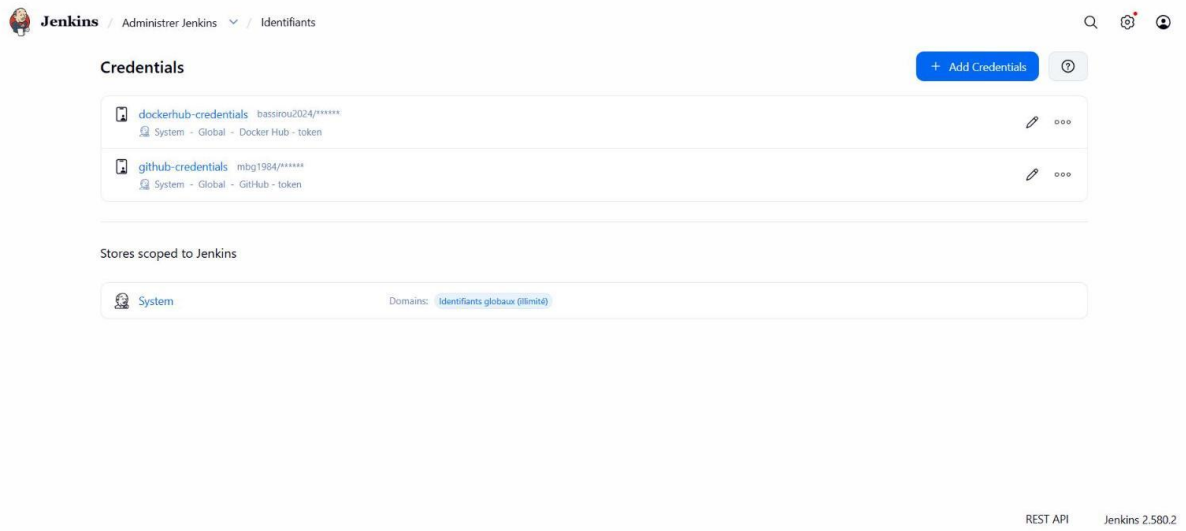


Figure 16 – Credentials enregistrés dans Jenkins

Ce que montre la capture : les credentials `dockerhub-credentials` (bassirou2024/****) **et `github-credentials` (mbg1984/**)Ce qu'elle prouve :** les tokens sont stockés chiffrés par Jenkins et n'apparaissent jamais en clair.

5.3 Configuration du job

Job de type **Pipeline**, *Pipeline script from SCM* : Git, `https://github.com/mbg1984/bijouterie-microservices.git`, branche `*/main`, script `Jenkinsfile`. Déclencheur : **scrutation de l'outil de gestion de version** (`H/2 * * * *`). Un webhook GitHub ne peut pas joindre un Jenkins qui tourne sur `localhost` sans tunnel : la scrutation toutes les 2 minutes est la solution adaptée en local.

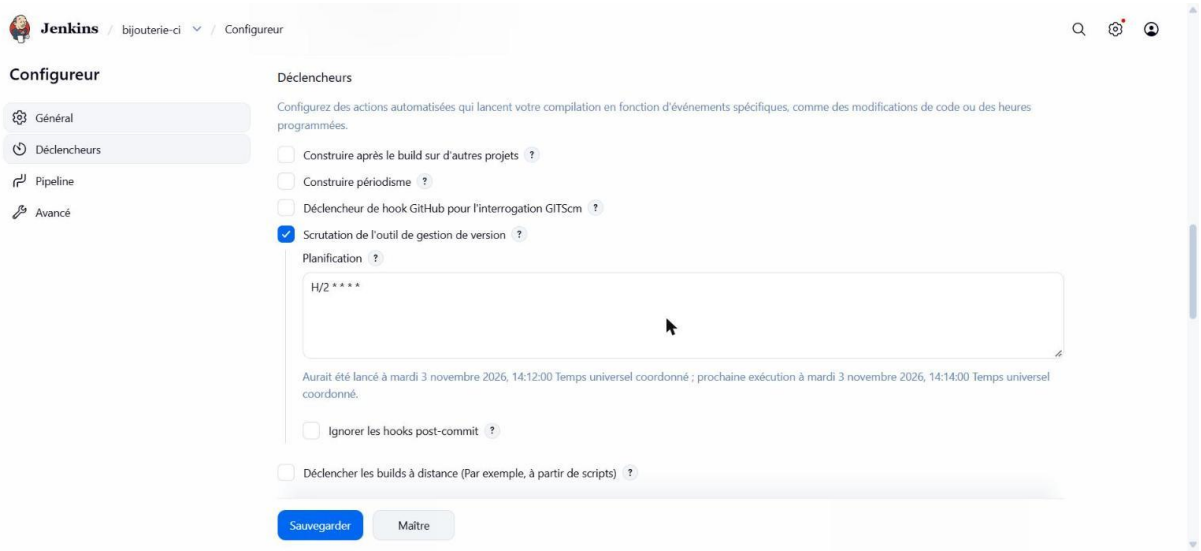


Figure 17 – Déclencheur du job : scrutation Git toutes les 2 minutes

Ce que montre la capture : la case « Scrutation de l'outil de gestion de version » cochée avec la planification `H/2 * * * *`. **Ce qu'elle prouve :** Jenkins détecte automatiquement les nouveaux commits.

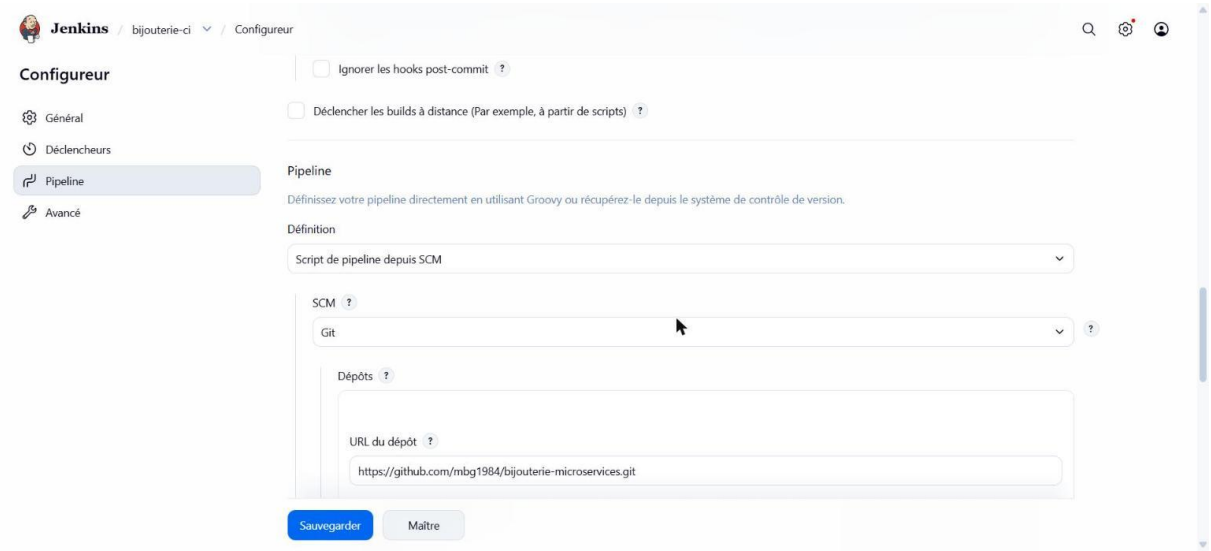


Figure 18 – Définition du pipeline depuis le SCM

Ce que montre la capture : « Script de pipeline depuis SCM », SCM Git et l'URL du dépôt GitHub. **Ce qu'elle prouve :** le Jenkinsfile est versionné avec le code : le pipeline est lui-même du code (*Pipeline as Code*).

5.4 Le Jenkinsfile commenté

Le pipeline déclaratif comporte les stages imposés **Checkout, Build, Tag, Push** et le stage bonus **Update manifests**. Points importants :

- **Double tag** : chaque image reçoit le tag immuable `<BUILD_NUMBER>` (utilisé dans les manifests) et `latest`.
- **Secrets** : `withCredentials` place le token dans des variables masquées (`****` dans la console) ; le script `sh` est entre guillemets simples `'''` pour que ce soit le shell, et non Groovy, qui lise le token ; `docker login --password-stdin` évite de passer le token en argument (visible dans la liste des processus).
- **GitOps** : le stage bonus remplace le tag dans `k8s/*-deployment.yaml` avec `sed`, puis committe et pousse.
- **Anti-boucle** : le commit de Jenkins contient `[skip ci]` ; lorsqu'un build est déclenché par la scrutation et que le dernier commit contient `[skip ci]`, les stages sont ignorés. Un build lancé à la main s'exécute toujours.
- `post { always { docker logout } }` supprime la session Docker Hub de l'agent après chaque build.

Jenkinsfile

```
// ============================================================================
// Pipeline CI de la Bijouterie
// ----------------------------------------------------------------------------
// Construit les 3 images Docker (frontend, backend, mysql), les tague avec le
// numéro de build Jenkins (+ latest), les pousse sur Docker Hub, puis
// (bonus GitOps) met à jour le tag des images dans k8s/*-deployment.yaml :
// Argo CD détecte ce commit et déploie la nouvelle version.
//
// Credentials Jenkins nécessaires (Manage Jenkins > Credentials) :
//   dockerhub-credentials : Username with password = compte Docker Hub + Access Token
//   github-credentials    : Username with password = compte GitHub + Personal Access Token
//                           (uniquement pour le stage « Update manifests »)
// ============================================================================
pipeline {
    // Exécution sur l'agent intégré (le conteneur Jenkins, qui a le client docker)
    agent any

    // Variables communes à tous les stages
```

```
    environment {
        DOCKERHUB_USER = 'bassirou2024'                    // compte Docker Hub
        IMAGE_PREFIX   = "${DOCKERHUB_USER}/bijouterie"    // -> bassirou2024/bijouterie-<service>
        IMAGE_TAG      = "${BUILD_NUMBER}"                 // tag unique = numéro du build
        GIT_REPO       = 'github.com/mbg1984/bijouterie-microservices.git'
    }

    // Paramètre pour activer / désactiver le stage bonus au lancement
    parameters {
        booleanParam(name: 'UPDATE_MANIFESTS', defaultValue: true,
                     description: 'Mettre à jour le tag des images dans k8s/ et pousser le commit
(GitOps)')
    }

    // Déclencheur : Jenkins interroge GitHub toutes les 2 minutes environ
    // (un webhook GitHub ne peut pas joindre un Jenkins local sans tunnel)
    triggers {
        pollSCM('H/2 * * * *')
    }

    options {
        timestamps()                                   // horodatage dans la console
        disableConcurrentBuilds()                      // un seul build à la fois
        buildDiscarder(logRotator(numToKeepStr: '10'))  // garde les 10 derniers builds
    }

    stages {

        // ------------------------------------------------------------------
        // 1. Checkout : récupération du code depuis le dépôt Git du job
        // ------------------------------------------------------------------
        stage('Checkout') {
            steps {
                checkout scm
                script {
                    // Le commit poussé par le stage « Update manifests » contient
                    // [skip ci] : on ne reconstruit pas les images pour ce commit,
                    // sinon chaque build déclencherait un nouveau build (boucle).
                    // Seuls les builds lancés par la scrutation Git (pollSCM) sont
                    // concernés : un build lancé à la main s'exécute toujours.
                    def message = sh(script: 'git log -1 --pretty=%s', returnStdout: true).trim()
                    def parScrutation =
!currentBuild.getBuildCauses('hudson.triggers.SCMTrigger$SCMTriggerCause').isEmpty()
                    env.SKIP_CI = (parScrutation && message.contains('[skip ci]')) ? 'true' : 'false'
                    echo "Dernier commit : ${message} (déclenché par Git = ${parScrutation}, skip ci =
${env.SKIP_CI})"
                }
            }
        }

        // ------------------------------------------------------------------
        // 2. Build : construction des 3 images à partir de leurs Dockerfiles
        // ------------------------------------------------------------------
        stage('Build') {
            when { expression { env.SKIP_CI != 'true' } }
            steps {
                sh '''
                    docker build -t bijouterie-frontend:${IMAGE_TAG} frontend/
                    docker build -t bijouterie-backend:${IMAGE_TAG}  backend/
                    docker build -t bijouterie-mysql:${IMAGE_TAG}    database/
                '''
            }
        }

        // ------------------------------------------------------------------
        // 3. Tag : chaque image reçoit 2 tags sur le dépôt Docker Hub
        //    - <numéro de build> : version immuable, utilisée dans les manifests
        //    - latest            : dernière version construite (jamais dans k8s/)
        // ------------------------------------------------------------------
        stage('Tag') {
            when { expression { env.SKIP_CI != 'true' } }
            steps {
                sh '''
```

```
                    for service in frontend backend mysql; do
                        docker tag bijouterie-${service}:${IMAGE_TAG} ${IMAGE_PREFIX}-
${service}:${IMAGE_TAG}
                        docker tag bijouterie-${service}:${IMAGE_TAG} ${IMAGE_PREFIX}-${service}:latest
                    done
                    docker images "${IMAGE_PREFIX}-*"
                '''
            }
        }

        // ---------------------------------------------------------------------
        // 4. Push : connexion à Docker Hub avec le credential Jenkins puis envoi
        //    Le token n'apparaît jamais en clair : withCredentials le place dans
        //    des variables masquées (****) dans la console.
        // ---------------------------------------------------------------------
        stage('Push') {
            when { expression { env.SKIP_CI != 'true' } }
            steps {
                withCredentials([usernamePassword(credentialsId: 'dockerhub-credentials',
                                                  usernameVariable: 'DOCKERHUB_LOGIN',
                                                  passwordVariable: 'DOCKERHUB_TOKEN')]) {
                    // Guillemets simples : c'est le shell (et non Groovy) qui lit
                    // le token -> il n'est pas interpolé dans le script Jenkins
                    sh '''
                        echo "$DOCKERHUB_TOKEN" | docker login -u "$DOCKERHUB_LOGIN" --password-stdin
                        for service in frontend backend mysql; do
                            docker push ${IMAGE_PREFIX}-${service}:${IMAGE_TAG}
                            docker push ${IMAGE_PREFIX}-${service}:latest
                        done
                    '''
                }
            }
        }

        // ---------------------------------------------------------------------
        // 5. (Bonus) Update manifests : remplace le tag des images dans
        //    k8s/*-deployment.yaml par le nouveau numéro de build, committe et
        //    pousse. Argo CD voit le commit et déploie la nouvelle version.
        // ---------------------------------------------------------------------
        stage('Update manifests') {
            when {
                allOf {
                    expression { env.SKIP_CI != 'true' }
                    expression { params.UPDATE_MANIFESTS }
                }
            }
            steps {
                withCredentials([usernamePassword(credentialsId: 'github-credentials',
                                                  usernameVariable: 'GIT_LOGIN',
                                                  passwordVariable: 'GIT_TOKEN')]) {
                    sh '''
                        # image: bassirou2024/bijouterie-<service>:<ancien tag> -> :<IMAGE_TAG>
                        sed -i -E "s#(image: ${IMAGE_PREFIX}-[a-z]+):[A-Za-z0-9._-]+#\\1:${IMAGE_TAG}#"
k8s/*-deployment.yaml
                        grep -n "image:" k8s/*-deployment.yaml

                        git config user.name  "Jenkins CI"
                        git config user.email "jenkins@bijouterie.local"
                        git add k8s/*-deployment.yaml
                        # Rien à committer si le tag était déjà le bon
                        git diff --cached --quiet && { echo "Manifests déjà à jour"; exit 0; }
                        git commit -m "ci: images bijouterie build ${IMAGE_TAG} [skip ci]"
                        git push "https://${GIT_LOGIN}:${GIT_TOKEN}@${GIT_REPO}" HEAD:main
                    '''
                }
            }
        }
    }

    // ---------------------------------------------------------------------
    // Actions exécutées après tous les stages, quel que soit le résultat
    // ---------------------------------------------------------------------
    post {
        always {
```

```
            sh 'docker logout || true'   // supprime la session Docker Hub de l'agent
        }
        success {
            echo "Images ${IMAGE_PREFIX}-{frontend,backend,mysql}:${IMAGE_TAG} publiées sur Docker Hub"
        }
    }
}
```

5.5 Résultat des builds

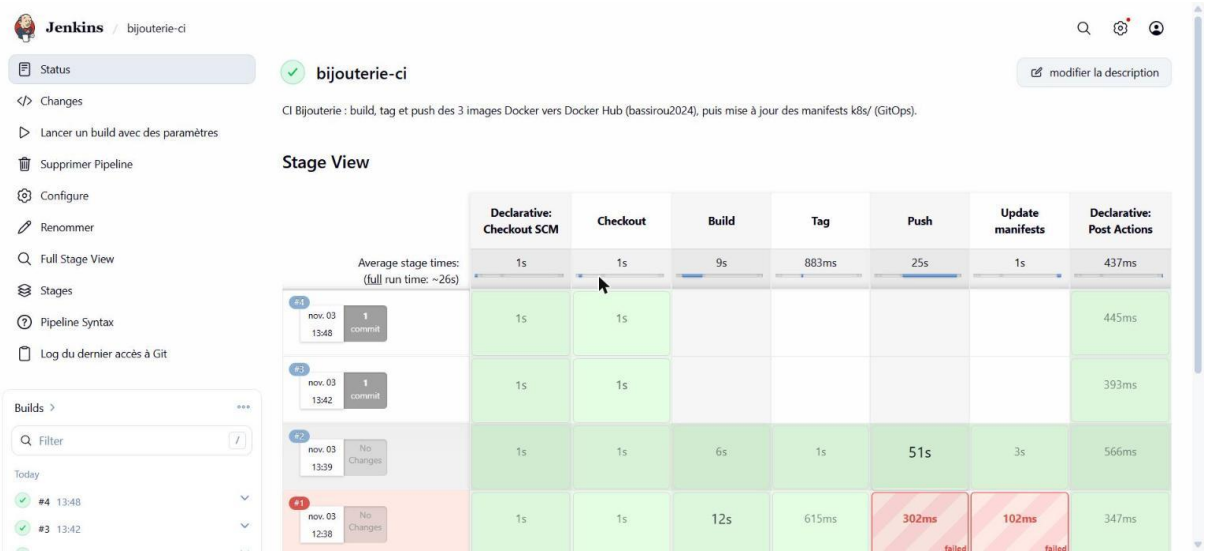


Figure 19 – Stage View du job bijouterie-ci

Ce que montre la capture : le build n°1 en échec sur Push (credentials pas encore créés), le build n°2 complet en vert, puis les builds n°3 et n°4 déclenchés par la scrutation qui s'arrêtent après Checkout. **Ce qu'elle prouve :** les stages demandés s'enchaînent, et la protection `[skip ci]` empêche Jenkins de se relancer sur ses propres commits.

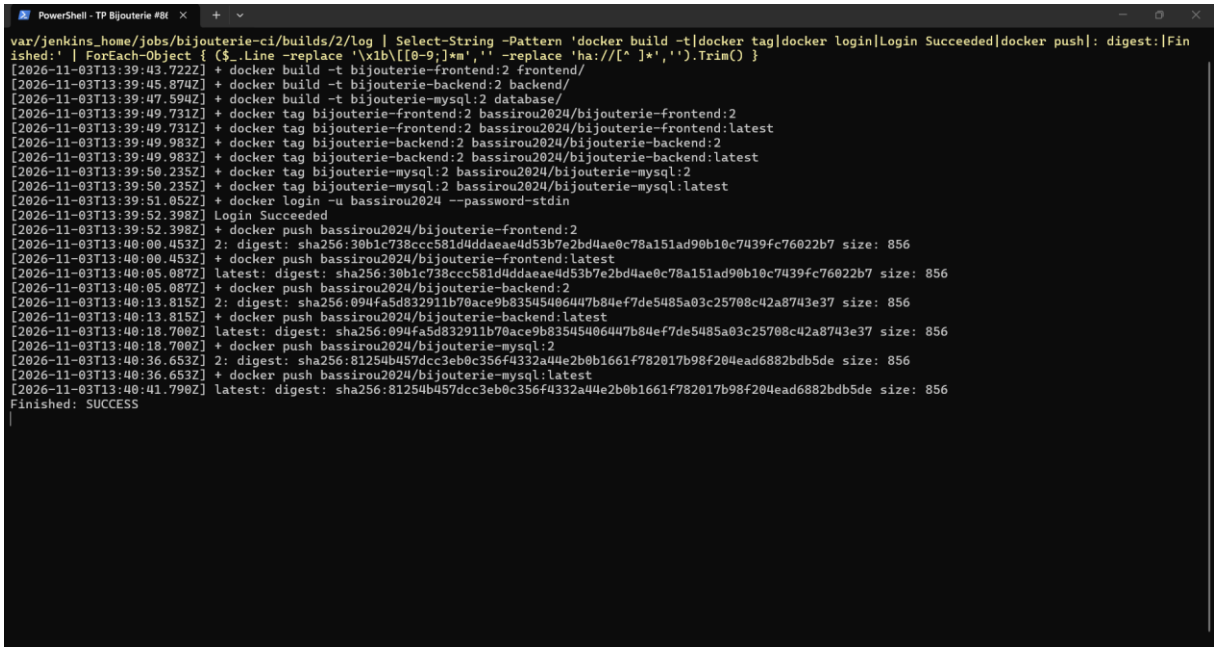


Figure 20 – Extrait de la console du build n°2

Ce que montre la capture : les commandes `docker build` des 3 images, les 6 `docker tag`, `Login Succeeded`, les 6 `docker push` avec leurs digests et `Finished: SUCCESS`. **Ce qu'elle prouve :** le build, le tag et la publication sur Docker Hub sont réalisés par Jenkins.

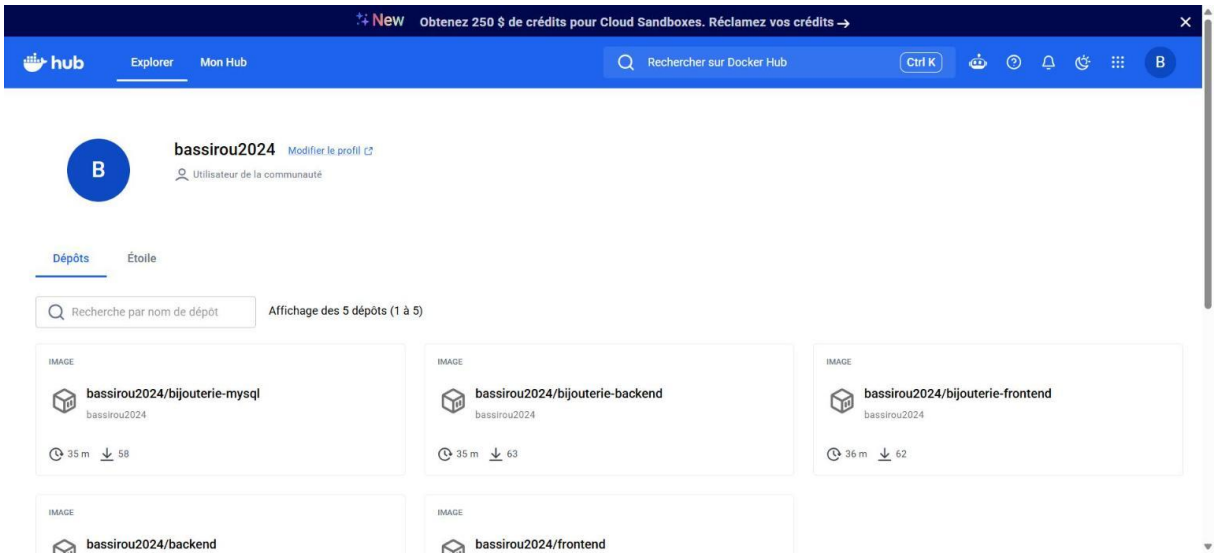


Figure 21 – Dépôts du compte Docker Hub bassirou2024

Ce que montre la capture : les dépôts `bijouterie-frontend`, `bijouterie-backend` et `bijouterie-mysql`. **Ce qu'elle prouve :** les trois images demandées existent sur mon compte Docker Hub.

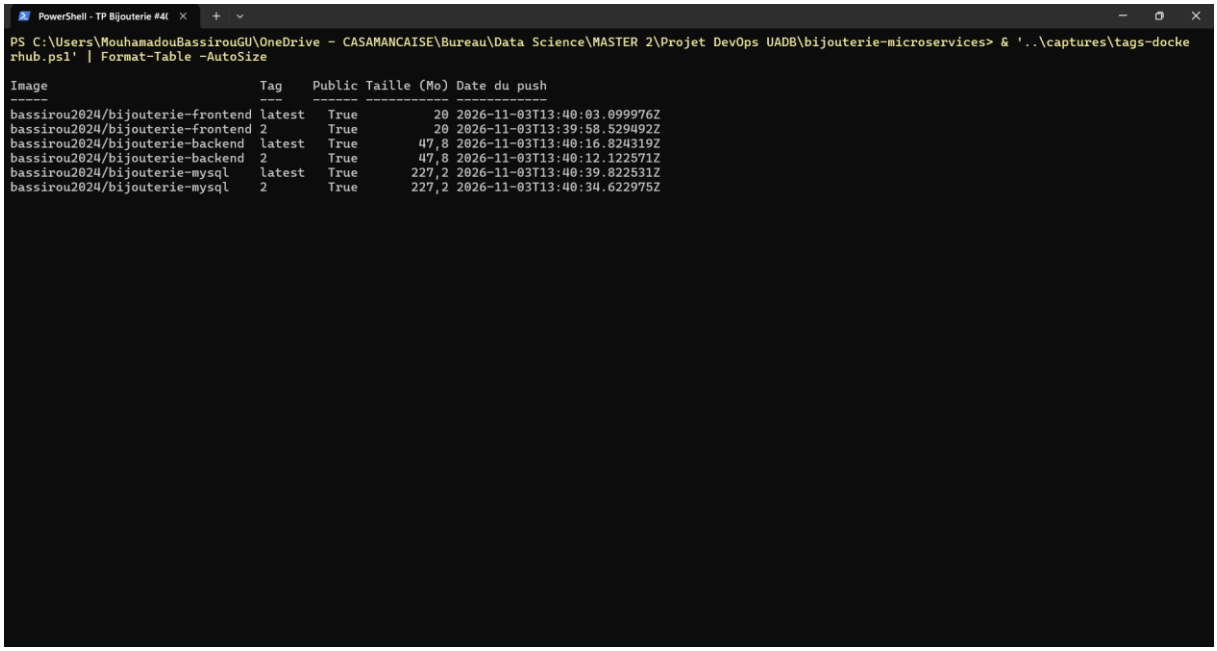


Figure 22 – Tags publiés (API Docker Hub)

Ce que montre la capture : pour chaque image, les tags `2` et `latest`, la visibilité publique (`True`) et la date de push. **Ce qu'elle prouve :** le double tag fonctionne et les images sont publiques, donc testables par l'enseignant.

6. Argo CD : déploiement continu GitOps

6.1 Installation

Commandes d'installation

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl port-forward svc/argocd-server -n argocd 8090:80
# mot de passe admin initial :
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d
```

Le dépôt étant public, il n'a pas été nécessaire de le déclarer avec un token dans Argo CD. Pour accéder à l'interface en HTTP en local, j'ai activé l'option `server.insecure` d'Argo CD (voir § 8).

6.2 L'Application bijouterie

argocd/application.yaml

```yaml
# ================================================================================
# Application Argo CD « bijouterie »
# --------------------------------------------------------------------------------
# Argo CD surveille le dossier k8s/ de la branche main du dépôt GitHub et
# applique son contenu dans le namespace bijouterie du cluster local.
# Git devient la seule source de vérité (GitOps) : plus de kubectl apply
# manuel sur le namespace bijouterie.
#
# Création (une seule fois) : kubectl apply -f argocd/application.yaml
# ================================================================================
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: bijouterie
  namespace: argocd          # les Applications vivent dans le namespace d'Argo CD
  # Si l'Application est supprimée, Argo CD supprime aussi les ressources
  # qu'elle a déployées (pods, services...) avant de disparaître
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default           # projet Argo CD par défaut (aucune restriction)

  # --- Source : où lire les manifests ---------------------------------------
  source:
    repoURL: https://github.com/mbg1984/bijouterie-microservices.git  # dépôt public
    targetRevision: main     # branche principale
    path: k8s                # dossier des manifests

  # --- Destination : où les déployer ----------------------------------------
  destination:
    server: https://kubernetes.default.svc   # le cluster où tourne Argo CD
    namespace: bijouterie                    # namespace par défaut des ressources

  # --- Politique de synchronisation -----------------------------------------
  syncPolicy:
    automated:               # synchronisation automatique à chaque commit
      prune: true            # supprime du cluster ce qui a été retiré de Git
      selfHeal: true         # annule les modifications faites à la main
                             # (ex. kubectl delete svc frontend -> recréé)
    syncOptions:
      - CreateNamespace=true # crée le namespace bijouterie s'il n'existe pas
```

- `source` : mon dépôt GitHub, branche `main`, dossier `k8s` ;
- `destination` : le cluster local `https://kubernetes.default.svc`, namespace `bijouterie` ;
- `automated` : synchronisation automatique à chaque commit ; `prune: true` supprime du cluster ce qui est retiré de Git ; `selfHeal: true` annule toute modification manuelle ;
- `CreateNamespace=true` : le namespace est créé s'il n'existe pas.

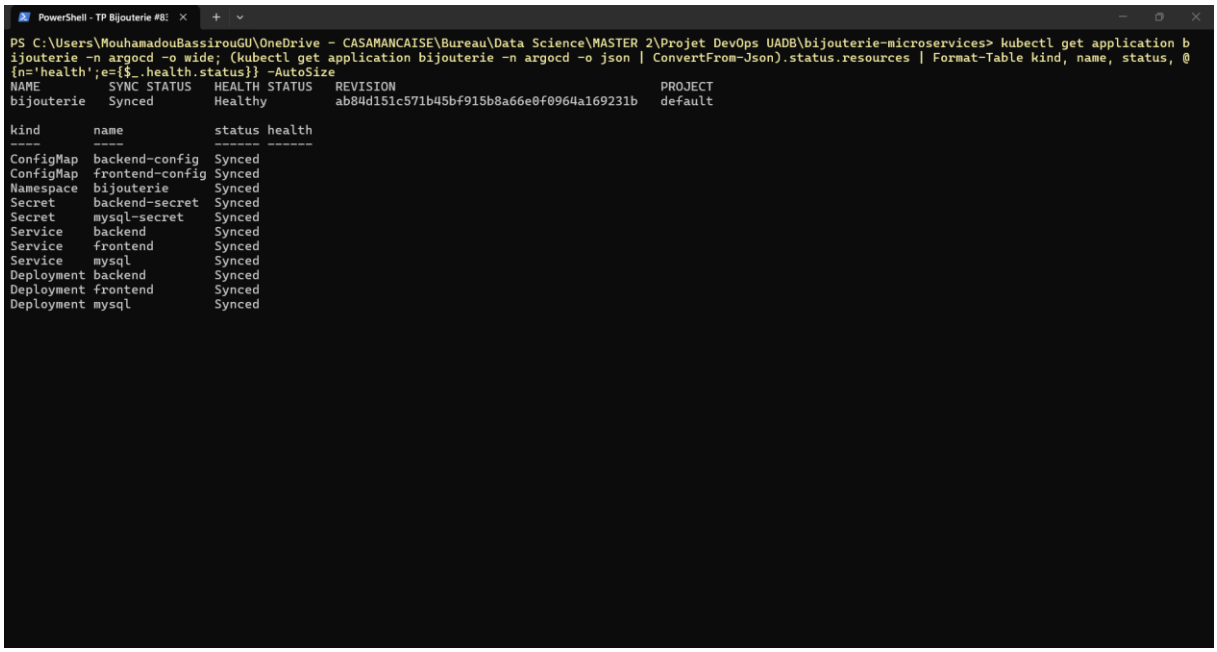


Figure 23 – État de l'Application (terminal)

Ce que montre la capture : `SYNC STATUS: Synced`, `HEALTH STATUS: Healthy` et les 11 ressources toutes `Synced`. **Ce qu'elle prouve :** le contenu du cluster est identique au contenu de Git.

6.3 Démonstrations

a) Modification par Git, sans kubectl

J'ai modifié deux lignes dans Git : `replicas: 2 → 3` pour le frontend et `SHOP_NAME: "Maison Amina Joaillerie"`, puis poussé le commit `be75400` à 14:21:05. Argo CD a détecté le commit et synchronisé seul à 14:22:30, sans aucun `kubectl apply`.

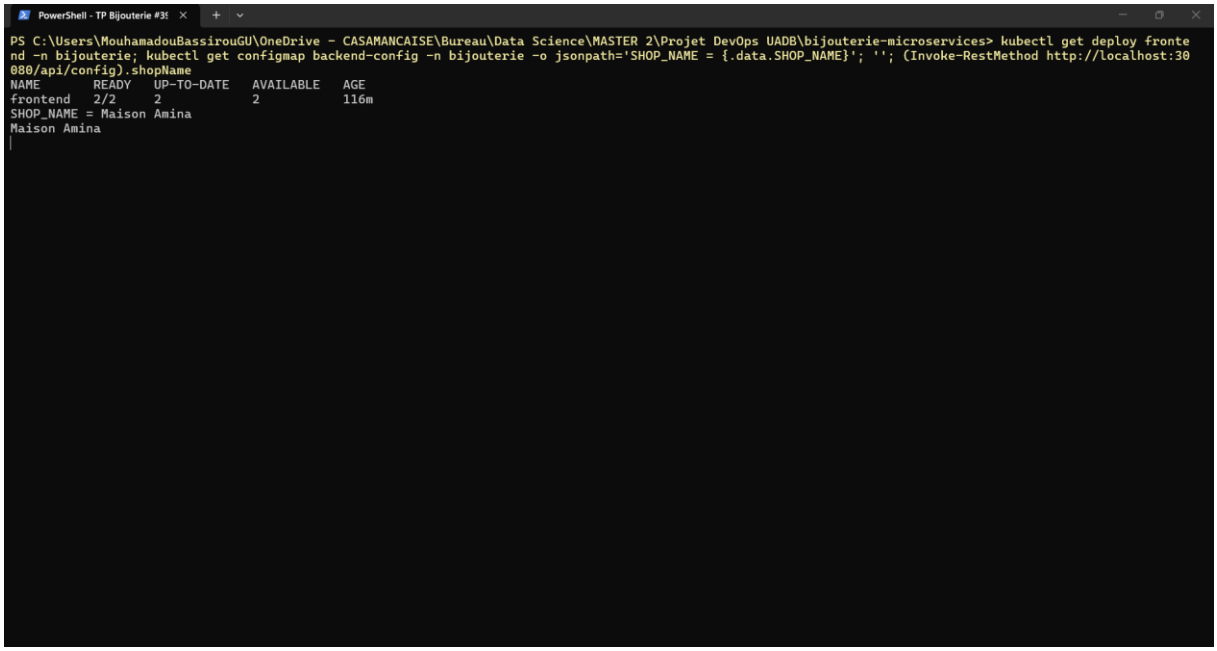


Figure 24 – AVANT le commit

Ce que montre la capture : le Deployment frontend à `2/2` et `SHOP_NAME = Maison Amina` (ConfigMap et API). **Ce qu'elle prouve :** l'état initial avant la modification Git.

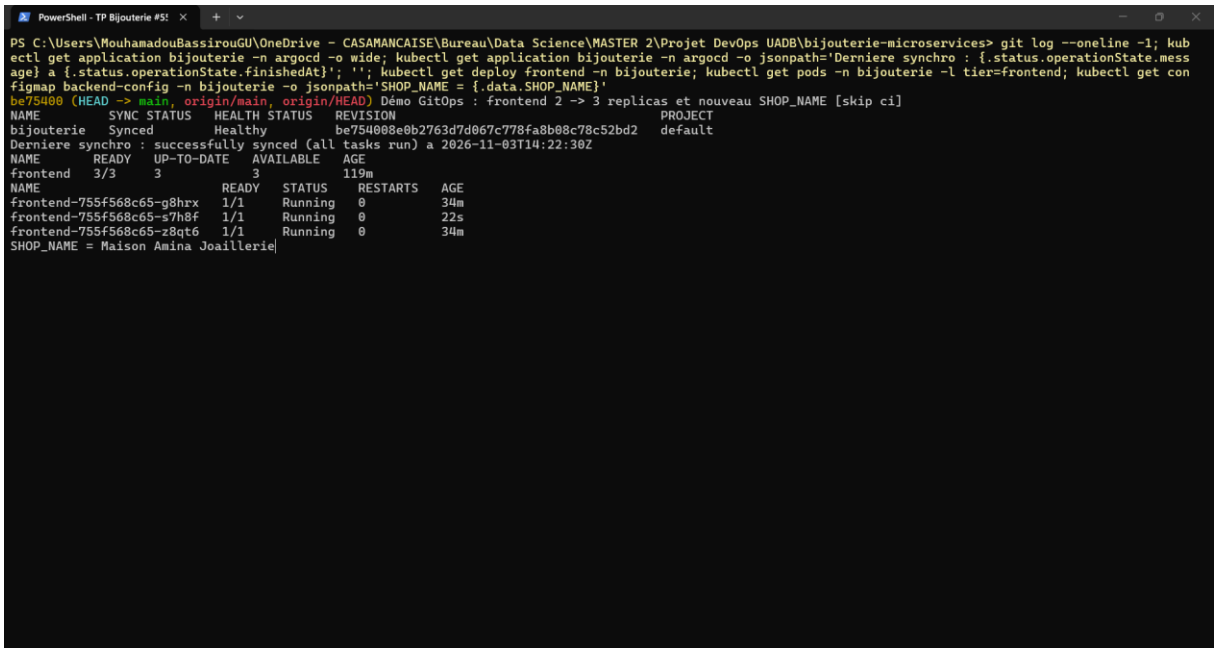


Figure 25 – APRÈS le commit : synchronisation automatique

Ce que montre la capture : le commit `be75400`, l'Application `Synced` sur cette révision, le message « successfully synced » à 14:22:30, le frontend à `3/3` et la nouvelle valeur de `SHOP_NAME`. **Ce qu'elle prouve :** Git est la seule source de vérité : un simple `git push` suffit à modifier le cluster.

b) Auto-réparation (self-heal)

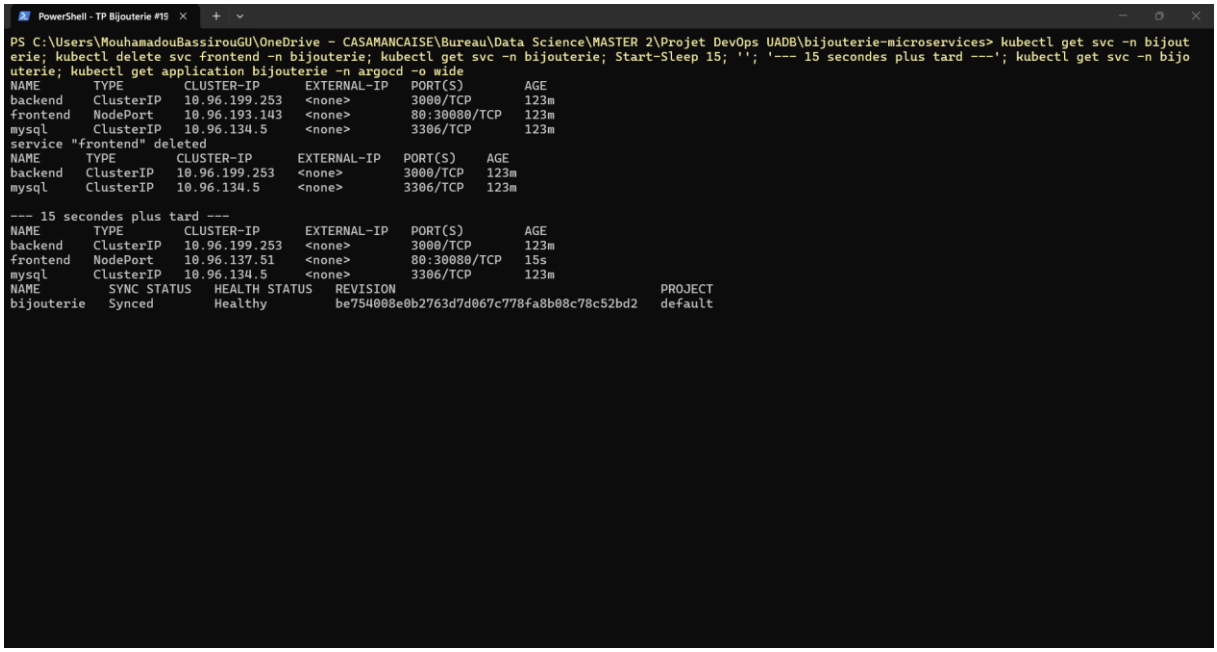


Figure 26 – Self-heal après suppression manuelle du Service frontend

Ce que montre la capture : `kubectl delete svc frontend` puis, 15 s plus tard, un Service `frontend` recréé (nouvelle ClusterIP `10.96.137.51`, âge 15 s) et l'Application toujours Synced / Healthy. **Ce qu'elle prouve :** Argo CD détecte l'écart avec Git et le corrige automatiquement.

c) Nouvelle version produite par Jenkins

Un build Jenkins (n°8) a construit et poussé les images taguées `8`, puis le stage « Update manifests » a commité `ba1e1ef ci: images bijouterie build 8 [skip ci]`. Argo CD a déployé cette révision automatiquement : les trois Deployments utilisent désormais les images `:8`.

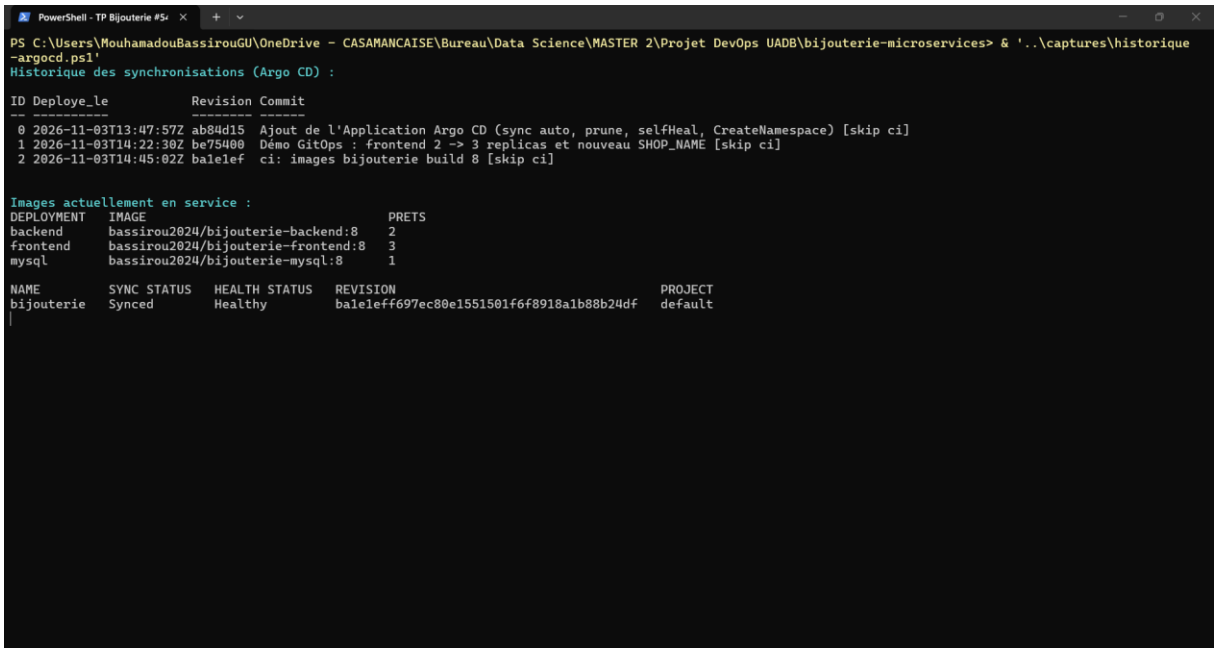


Figure 27 – Historique des synchronisations Argo CD

Ce que montre la capture : les 3 déploiements successifs (ab84d15 : création, be75400 : démo GitOps, ba1e1ef : build 8) et les images `:8` en service. **Ce qu'elle prouve :** la chaîne complète Jenkins → Git → Argo CD → cluster fonctionne sans intervention manuelle.

7. Sécurité : la gestion des Secrets dans Git

Pourquoi c'est une mauvaise pratique en production

Un Secret Kubernetes n'est qu'**encodé en base64**, pas chiffré : `echo 'bWEtdmFsZXVy' | base64 -d` redonne immédiatement `ma-valeur`. Toute personne ayant accès au dépôt (ou à un fork, une copie, une sauvegarde) lit les mots de passe.

L'**historique Git est permanent** : même supprimé plus tard, le secret reste dans les anciens commits.

Le dépôt est **public** pour ce TP : les identifiants MySQL sont donc publics. Ils ne protègent ici qu'une base de démonstration locale, non exposée (ClusterIP).

Ce qu'il faudrait faire en production

Solution	Principe
Sealed Secrets (Bitnami)	on committe un `SealedSecret` chiffré avec la clé publique du cluster ; seul le contrôleur peut le déchiffrer
SOPS + age / KMS	les valeurs du YAML sont chiffrées dans Git ; Argo CD les déchiffre via un plugin (KSOPS)
External Secrets Operator	Git ne contient qu'une référence ; les valeurs viennent de Vault, AWS Secrets Manager, Azure Key Vault…
HashiCorp Vault + Argo CD Vault Plugin	remplacement des valeurs au moment de la synchronisation
Durcissement du cluster	chiffrement d'etcd au repos, RBAC limitant `get secret`, rotation régulière des mots de passe

Bonnes pratiques appliquées dans ce TP

- aucun mot de passe en clair dans les Deployments ni dans le Jenkinsfile ;
- tokens Docker Hub et GitHub à **portée minimale** (un seul dépôt, Contents seulement) et **expiration** à 30 jours ;
- tokens masqués dans Jenkins (`****`) et `docker logout` après chaque build ;
- fichier `.env` local exclu de Git par `.gitignore` ;
- aucun mot de passe ni token visible sur les captures de ce rapport ;
- `server.insecure` d'Argo CD réservé à un accès local via port-forward : en production, Argo CD serait exposé en HTTPS derrière un Ingress avec un certificat valide.

8. Difficultés rencontrées et solutions

Mémoire de Docker Desktop insuffisante

Symptôme : Docker Desktop (moteur Hyper-V) ne disposait que de 2 Go : `docker ps` ne répondait plus et la création du cluster kind restait bloquée.

Diagnostic et solution : Passage de la mémoire à 8 Go puis 6 Go : échec de Hyper-V « DockerDesktopVM ne parvient pas à allouer 8192 Mo de mémoire RAM : Ressources système insuffisantes (0x800705AA) ». Diagnostic : seulement 2,1 Go de RAM libre, occupée par les navigateurs. Solution : fermer les applications et fixer **4 Go / 4 CPU**.

Fichier de configuration de Docker corrompu

Symptôme : Après modification de `settings-store.json` avec PowerShell 5.1, Docker refusait de démarrer : « invalid character 'ï' looking for beginning of value ».

Diagnostic et solution : Cause : PowerShell 5.1 ajoute un BOM UTF-8. Le fichier a été réécrit en UTF-8 sans BOM.

Port 8080 déjà utilisé

Symptôme : `docker compose up` : « Ports are not available: exposing port TCP 0.0.0.0:8080 … bind ».

Diagnostic et solution : `Get-NetTCPConnection -LocalPort 8080` → processus `java`. Solution : `FRONTEND_PORT=8082` dans `.env`.

Premier push GitHub refusé

Symptôme : « Invalid username or token. Password authentication is not supported for Git operations ».

Diagnostic et solution : GitHub n'accepte plus les mots de passe. Authentification via Git Credential Manager (connexion par navigateur).

Build Jenkins n°1 en échec

Symptôme : « ERROR: Could not find credentials entry with ID 'dockerhub-credentials' » au stage Push.

Diagnostic et solution : Création des credentials `dockerhub-credentials` puis `github-credentials` ; le build n°2 est passé en SUCCESS.

Boucle de builds et règle [skip ci]

Symptôme : Le commit poussé par Jenkins relançait Jenkins. Ma première règle `[skip ci]` ignorait aussi un build lancé à la main lorsque le dernier commit contenait `[skip ci]` (builds n°5 et 6 vides).

Diagnostic et solution : La règle ne s'applique plus qu'aux builds causés par la scrutation (`SCMTriggerCause`). Leçon : le message de commit de la correction contenait lui-même « [skip ci] », ce qui a fait ignorer le build n°7.

Interface Argo CD en HTTPS

Symptôme : Certificat auto-signé refusé par le navigateur intégré utilisé pour les captures.

Diagnostic et solution : Activation de `server.insecure: "true"` dans `argocd-cmd-params-cm` et port-forward sur le port 80 (usage local uniquement).

Captures traduites automatiquement

Symptôme : Le navigateur traduisait les pages anglaises (« latest » → « dernier »), ce qui rendait certaines captures trompeuses.

Diagnostic et solution : Captures des preuves sensibles (log Jenkins, tags Docker Hub, état Argo CD) prises directement dans le terminal.

9. Conclusion

L'application « Bijouterie » est déployée sur un cluster Kubernetes local avec une configuration entièrement externalisée (2 ConfigMaps, 2 Secrets), des Services adaptés à chaque besoin (NodePort pour le site, ClusterIP pour l'API et la base) et des sondes de santé. Jenkins construit, tague deux fois et publie les trois images sur Docker Hub, puis met à jour les manifests. Argo CD applique automatiquement chaque commit, répare les modifications manuelles et garde l'historique des versions.

Ce TP m'a permis de comprendre concrètement le principe GitOps : après la mise en place d'Argo CD, plus aucun `kubectl apply` n'a été fait à la main sur le namespace `bijouterie` ; un `git push` suffit à faire évoluer l'application, de façon traçable et réversible.

Pistes d'amélioration

- rendre MySQL persistant avec un **PersistentVolumeClaim** (bonus) ;
- monter `init.sql` via une **ConfigMap** et utiliser l'image officielle `mysql:8.4` (bonus) ;
- remplacer les Secrets en base64 par **Sealed Secrets** ou **External Secrets** ;
- ajouter des étapes de tests et de scan de vulnérabilités (Trivy) dans le pipeline ;
- exposer le site avec un **Ingress** et un certificat TLS plutôt qu'un NodePort.